

Algorithm Specification and Supporting Documentation

LOOM: An Authenticated Key Exchange

Long Chen¹, Zidi Zhuang¹, Yuyang Xiao¹, Cong Zhang², Si Gao¹, Jipeng Zhang³, Guofeng Tang⁴, Zihan Liu¹, Yifan Ming¹, Jianhua Hou⁵, Ruiqi Mi¹, Tanping Zhou⁶, Pengfei Chen², Yutao Sun⁷, Shuo Chen⁶, Yingjie Hao¹, Taiyu Wang², and Minxuan Mei²

¹Institute of Software, Chinese Academy of Sciences, China

²Zhejiang University, China

³National University of Singapore, Singapore

⁴Singapore Management University, Singapore

⁵Information Engineering University, China

⁶Engineering University of PAP, China

⁷Beijing Electronic Science and Technology Institute, China

June 30, 2026

Abstract

This document specifies LOOM, a compact post-quantum Authenticated Key Exchange (AKE) primitive submitted to the New Generation Commercial Cryptography (NGCC) algorithm call by ICCS. LOOM investigates an imperfect-correctness design for post-quantum AKE: instead of requiring negligible decryption failure probability, it permits a small and explicitly detectable Decryption Failure Rate (DFR) in exchange for compact KEM payloads and highly efficient decapsulation. Its ephemeral key exchange is performed by a dedicated LOOM-KEM, which is analyzed under Indistinguishability under Chosen Plaintext Attack with Failure (IND-CPAF) together with decapsulation rigidity, ensuring that decryption errors result in an explicit failure symbol rather than a malformed session key. This enables the removal of the re-encryption step used in Fujisaki–Okamoto-style transforms, improving decapsulation efficiency while supporting a proof of session-key security in the post-specified-peer Canetti–Krawczyk model.

Architecturally, LOOM adopts a modular Sign-and-MAC design that decouples ephemeral key exchange from long-term identity authentication. This structure provides three key system-level advantages: (1) **MTU-Aware DoS Mitigation**: By keeping the initial LOOM-KEM payloads compact, the unauthenticated first-round flights remain strictly below standard network MTU limits, eliminating IP-layer fragmentation and inherently neutralizing fragmentation-based reassembly-buffer Denial-of-Service (DoS) attacks. (2) **Seamless PKI Compatibility**: By employing long-term signatures (instantiated via Shuttle) for authentication, LOOM natively integrates with existing CA and X.509 certificate infrastructures without requiring a disruptive transition to static KEM certificates. (3) **Robust Perfect Forward Secrecy (PFS)**: The long-term signing keys are strictly isolated from the symmetric key derivation process, ensuring that future credential compromises provide an adversary with zero mathematical leverage to decrypt past recorded traffic.

Overall, LOOM demonstrates that controlled decryption failures can be safely incorporated into post-quantum authenticated key exchange to achieve a net gain in both cryptographic efficiency and real-world network robustness.

Contents

1	Introduction	3
1.1	Main Features and Advantages	4
2	Algorithm Description	5
2.1	Preliminaries and Notation	6
2.2	Primitive Specification	6
2.2.1	AKE.KeyGen	7
2.2.2	AKE.Init	7
2.2.3	AKE.Respond	7
2.2.4	AKE.AuthInit (Shared Secret Derivation & Failure Handling)	7
2.2.5	AKE.AuthResp	8
2.2.6	AKE.Confirm	8
2.2.7	KEM components	9
2.2.8	Signature Component	12
2.3	Symmetric Primitives and Quantum Security Levels	12
2.4	Parameter Sets and Suite Composition	16
3	Design Rationale	17
3.1	Underlying Hard Problems	17
3.2	Parameter Selection Strategy	17
3.3	Decryption Failure Rate (DFR) Analysis	19
3.4	Architectural Paradigm: Sign-and-MAC vs. Multi-KEM	20
4	Security Statement and Analysis	22
4.1	Theoretical Security	22
4.1.1	Hardness Assumptions	22
4.1.2	Security Reduction of LOOM-CPAPKE	23
4.1.3	IND-CPAF and Rigidity KEM	26
4.1.4	The Security Model: Canetti-Krawczyk (CK)	30
4.1.5	Security Reduction of LOOM-AKE	32
4.2	Concrete Security Estimates	43
4.2.1	Security Analysis of the KEM Component	44
4.2.2	Security Analysis of the SIG Component	44
4.2.3	Summary	46
4.3	Side-Channel Resistance and Implementation Security	47
4.4	Other Security Properties	47
5	Performance Evaluation and Implementation Analysis	48
5.1	Evaluation Methodology	49
5.2	Software Performance on x86 Processors	49
5.2.1	LOOM-KEM: Reference vs. AVX2	49
5.2.2	Shuttle signature: Reference vs. AVX2	50
5.2.3	Loom-AKE per-algorithm cost	50
5.2.4	End-to-end session cost	50
5.3	Optimization Techniques	51
5.3.1	Vectorization with AVX2 (x86_64)	51
5.4	Transmission and Storage Overhead	53

1 Introduction

The transition to Post-Quantum Cryptography (PQC) in network protocols faces a practical deployment challenge: lattice-based key exchange mechanisms typically have substantially larger public keys and ciphertexts than their classical counterparts [11, 7]. For UDP-based protocols such as the Internet Key Exchange version 2 (IKEv2) [36], this communication overhead is not merely a performance concern. The initial key exchange messages are sent before application-layer fragmentation can be protected by established keys, and oversized packets must therefore fall back to IP-layer fragmentation or require additional protocol mechanisms such as `IKE_INTERMEDIATE` [45].

As analyzed by Hulsing et al. [34], relying on IP-layer fragmentation during the unauthenticated handshake phase is a security and operational disaster. First, it exposes the responder to devastating Denial-of-Service (DoS) attacks. Because the incoming fragments cannot be cryptographically authenticated before reassembly, an off-path adversary can flood the responder with incomplete, spoofed IP fragments. This forces the responder to allocate and hold state in its IP reassembly buffers for arbitrary, unauthenticated garbage, rapidly exhausting system memory and blocking legitimate connections. Second, IP fragmentation is notoriously fragile in practice; middleboxes, firewalls, and NAT devices frequently filter or drop non-initial IP fragments due to security policies or limited state-tracking capabilities [10]. In IPv6, where the minimum link MTU is 1280 bytes [20] and routers do not perform fragmentation, any packet exceeding this limit is highly susceptible to being dropped entirely. Even a single lost fragment forces the retransmission of the entire IP packet, exponentially compounding handshake latency under lossy network. Consequently, ensuring that the initial key exchange payloads fit within a single, unfragmented UDP packet (ideally within the 1280-byte IPv6 MTU limit) is not just a performance optimization, but a fundamental security requirement to mitigate fragmentation-based DoS and guarantee protocol availability.

LOOM is a lattice-based Authenticated Key Exchange (AKE) primitive submitted to the New Generation Commercial Cryptography (NGCC) call. Its design follows the *Dance with Noise* philosophy: instead of enforcing perfect correctness at all costs, LOOM permits a small and explicitly detectable Decryption Failure Rate (DFR) in order to obtain aggressive payload compression and efficient decapsulation. This design is particularly attractive for ephemeral AKE settings, where a fresh KEM key pair is used in each session and the observable failure event can be incorporated into the security model.

Concretely, LOOM combines two compact lattice-based components:

- **Compact IND-CPAF KEM.** The ephemeral key exchange is performed by a dedicated, specially designed KEM, which we designate as LOOM-KEM. It is required to satisfy *Indistinguishability under Chosen Plaintext Attack with Failure (IND-CPAF)* together with *Rigidity*. This security target is weaker than full IND-CCA security but is sufficient for our all-ephemeral AKE construction. It allows LOOM-KEM to completely avoid the expensive ciphertext re-encryption step used in standard Fujisaki–Okamoto-style transforms [23, 29]. LOOM-KEM is built upon an optimized MLWR core, using tailored compact encoding and compression techniques to minimize the size of the transmitted KEM material. At the 256-bit classical security level, the KEM public key and ciphertext are 1312 and 1344 bytes respectively (Table 4).
- **Compact signature from Shuttle.** Mutual authentication is provided by Shuttle [16], a lattice-based digital signature scheme from the Fiat–Shamir with aborts family. Shuttle achieves compact authentication material through iterative rejection sampling and structural compression. In LOOM, the signature component is used as a black-box existentially unforgeable signature scheme for binding peer identities to the key-exchange transcript.

A central issue in using an imperfect-correctness KEM is that observable protocol aborts may

act as a Failure Oracle. Such decryption-failure leakage has long been recognized as a serious attack vector in lattice-based and NTRU-like schemes [33, 19, 26, 27]. Recent works further study how decryption failures can be handled or proven secure under appropriate models [31, 40]. To address this issue, LOOM adopts the IND-CPAF model and enforces *rigid* decapsulation. In particular, every decapsulation error is deterministically detected: the decapsulation algorithm outputs a dedicated failure symbol $\perp$ instead of producing a malformed or biased session key. In the ephemeral AKE setting, the adversary can observe at most one failure event per KEM key pair. IND-CPAF security captures this bounded leakage and ensures that, conditioned on successful completion, the derived session key remains indistinguishable from random. This enables a proof of SK-security in the post-specified-peer variant of the Canetti–Krawczyk (CK) model [12], despite a controlled DFR from approximately $2^{-18.9}$ to $2^{-24.8}$ (Table 4).

Although LOOM is specified as a cryptographic AKE primitive rather than a complete network protocol, its compact KEM payloads make it well suited for UDP-based deployment profiles. In particular, the 128-bit instance can be integrated into the existing IKEv2 `IKE_SA_INIT` exchange under the measured IKEv2 overhead while remaining below the IPv6 minimum MTU. Specifically, with a worst-case measured non-KEM IKEv2 overhead of 241 bytes and an IPv6/UDP overhead of 48 bytes, the largest first-exchange packet carrying a 752-byte KEM payload has size

$$48 + 241 + 752 = 1041 \text{ bytes,}$$

which is below the 1280-byte IPv6 minimum MTU [20]. Larger parameter sets target MTU-aware fixed-suite deployment profiles rather than unmodified general-purpose IKEv2. This separation follows the same architectural observation behind IKEv2 message fragmentation [44]: fragmentation before key establishment is fundamentally different from authenticated fragmentation after key establishment.

By combining compact LOOM-KEM key exchange, Shuttle-based authentication, explicit failure detection, and the IND-CPAF security framework, LOOM demonstrates that imperfect-correctness KEMs can be safely composed into authenticated key exchange protocols while providing substantial bandwidth and decapsulation-efficiency advantages.

1.1 Main Features and Advantages

Unlike traditional post-quantum key exchange designs, LOOM-AKE is specifically engineered to resolve the conflict between lattice payload inflation and real-world network constraints. It distinguishes itself through three interconnected architectural contributions:

- **MTU-Aware Scheduling and Mitigation of Fragmentation-Based DoS.** Lattice-based primitives typically suffer from large public keys and ciphertexts, which trigger IP-layer fragmentation during the initial handshake. This exposes the responder to severe reassembly-buffer Denial-of-Service (DoS) attacks, where an off-path attacker floods the responder with incomplete, forged fragments. LOOM-AKE inherently neutralizes this threat through its two-tiered, MTU-aware scheduling:
 - At the 128-bit security level, LOOM uses a 752-byte KEM public key and a 736-byte ciphertext. This allows the initial exchange to remain comfortably below the 1280-byte IPv6 minimum MTU [20], avoiding IP fragmentation entirely without requiring additional round trips (such as `IKE_INTERMEDIATE` [45]).
 - At the 256-bit security level, the initial KEM flights are kept unfragmented under the standard 1500-byte MTU. The larger authentication payloads (signatures and certificates) are delayed to flights 3 and 4, allowing them to be fragmented at the application layer and authenticated under KEM-derived keys. This prevents unauthenticated fragmentation and eliminates the associated DoS attack vector.

- **Modular Sign-and-MAC Design for Seamless PKI Integration.** Instead of adopting a Multi-KEM approach that requires CAs to issue static KEM-key certificates (disrupting the entire global PKI ecosystem), LOOM-AKE decouples ephemeral key exchange from long-term identity authentication. By employing a modular Sign-and-MAC structure (using ephemeral LOOM-KEM and long-term Shuttle signatures), LOOM-AKE natively preserves the role of digital signatures for identity verification. This allows immediate compatibility with existing X.509 certificate chains and enables algorithm-independent agility—allowing the underlying signature or KEM components to be upgraded independently without invalidating the high-level security reduction.
- **Robust Perfect Forward Secrecy (PFS) via Key Isolation.** In LOOM-AKE, the long-term signature key is strictly isolated from the symmetric key derivation process; its sole purpose is to sign the role-separated handshake transcripts ($DTBS_I$ and $DTBS_R$). The ephemeral KEM key pair is generated freshly for each session and erased immediately after use. Because the long-term signing key is never used to encrypt or decrypt keying material, a future compromise of the long-term credential provides the adversary with zero mathematical leverage to decrypt past recorded traffic. This guarantees robust, physical-level Perfect Forward Secrecy.

Cryptographic Failures vs. Network Packet Loss. A common skepticism among cryptographers is that allowing any non-zero decryption failure rate (DFR) inherently compromises system-level reliability. We counter this perspective by contextualizing cryptographic failures within real-world network transport dynamics and existing post-quantum protocol designs.

The paradigm of trading a minor, non-negligible DFR for substantial bandwidth savings is well-established in state-of-the-art post-quantum protocols. A prominent example is *Post-Quantum WireGuard* [34] (IEEE S&P 21'), where the authors design the *Dagger* KEM (a compressed variant of Saber) with a DFR of $2^{-25.25}$ to strictly fit the handshake into the 1280-byte IPv6 MTU. Their system evaluations demonstrated that such a microscopic failure rate has a negligible impact on user experience and is completely masked by standard network anomalies.

In standard UDP-based transport layers (such as those carrying IKEv2 or WireGuard), network-layer packet loss is an unavoidable and frequent event. Empirical network measurements show that average packet loss rates on the public Internet typically range from 0.1% to over 3.0%, and can spike significantly under congested or intercontinental links. In stark contrast, our aggressively compressed KEM parameter sets restrict the DFR to a range between 2^{-19} and 2^{-25} (i.e., 0.00019% to 0.000003%). This means that a cryptographic decapsulation failure is at least *three to five orders of magnitude less likely* than a standard network packet drop.

Conversely, insisting on perfect correctness (e.g., using standard ML-KEM) inflates packet sizes and forces IP-layer fragmentation. In lossy or restrictive network environments, IP fragmentation is notorious for causing high packet drop rates (often exceeding 3% $\sim$ 5%) due to middleboxes blocking non-initial fragments or the multiplicative effect of fragment loss. Therefore, trading a mathematically bounded, microscopic DFR (which can be seamlessly handled by a simple protocol-level retry) for a fragmentation-free packet footprint is not only secure under our IND-CPAF framework, but also yields a net gain in overall system-level reliability and robustness.

2 Algorithm Description

This section specifies the proposed Authenticated Key Exchange (LOOM-AKE) primitive. Unlike a full network protocol (which includes application-layer AEAD encryption and packet formatting), the LOOM-AKE is defined strictly as a foundational cryptographic primitive. It is

constructed via a black-box composition of an IND-CPAF secure Key Encapsulation Mechanism (KEM) and a EUF-CMA-secure Digital Signature scheme (SIG).

The core innovation of this AKE primitive is its ability to safely manage minor decryption failures (via the KEM’s Rigidity and IND-CPAF security) to achieve highly compact communication sizes, making it ideal for fragmentation-free integration into real-world protocols like IKEv2.

2.1 Preliminaries and Notation

We define the cryptographic primitives and basic notations used in the AKE primitive computations. To avoid ambiguity, Table 1 fixes the canonical name, input/output interface, and intended role of every cryptographic component and message used in the algorithm specification. All subsequent sections, including the security analysis, use these symbols and interfaces.

Table 1: Normative symbols and interfaces for LOOM-AKE.

Component / symbol	Canonical interface and role
(KEM components)	
PRF	expands secret seed $seed_{sk}$ into the secret vectors $\mathbf{s}, \mathbf{r}$ and drives the randomized lifting.
XOF _{pub}	expands the public seed $seed_A$ into the public matrix $\hat{\mathbf{A}}$.
XOF	splits the master seed via $(seed_A, seed_{sk}) = \text{XOF}(seed_0)$, where $seed_{sk}$ is secret.
F	$t \leftarrow \mathbf{F}_m(c)$: the KEM rigidity (plaintext-verification) tag, a PRF keyed by the encapsulated message m and evaluated on the PKE ciphertext c .
G	$K \leftarrow \mathbf{G}(m)$: derives the raw KEM shared secret from the message m .
(ek, dk)	ephemeral KEM public/secret key pair (per session).
$ct_{kem} = (c \parallel t)$	KEM ciphertext: PKE ciphertext c concatenated with rigidity tag t .
K	raw KEM shared secret.
KDF ⁺	$(SS, mk_I, mk_R) \leftarrow \text{KDF}^+(k, N_I \parallel N_R)$: derives the session secret SS and the two MAC keys mk_I, mk_R from the salt $N_I \parallel N_R$ and the KEM shared secret K .
MAC	$\tau \leftarrow \text{MAC}(mk, msg)$: produces an authentication tag under key mk . Used as $\tau_I = \text{MAC}(mk_I, ID_I)$ and $\tau_R = \text{MAC}(mk_R, ID_R)$.
SS	final derived session secret.
mk_I, mk_R	MAC keys of $\mathcal{I}$ and $\mathcal{R}$.
(vk_X, sk_X)	static signature verification/signing keys of party $X \in \{I, R\}$.
N_I, N_R	nonces of $\mathcal{I}$ and $\mathcal{R}$.
$s_I, s_R, s = s_I \parallel s_R$	local SPIs and the complete session identifier.
ID_I, ID_R	the identity strings of the Initiator $\mathcal{I}$ and Responder $\mathcal{R}$
$msg_{I,1}, msg_{R,1}, msg_{I,2}, msg_{R,2}$	the four protocol messages, in order of transmission.

2.2 Primitive Specification

The LOOM-AKE primitive is defined by six core computational algorithms: AKE.KeyGen, AKE.Init, AKE.Respond, AKE.AuthInit, AKE.AuthResp, and AKE.Confirm. These algorithms cover the initialization, message generation, mutual authentication, and shared secret derivation required by the LOOM standards.

2.2.1 AKE.KeyGen

Generates the long-term identity keys for the participating entities.

Algorithm 1 AKE.KeyGen(1^λ)

- 1: **Output:** Long-term public key vk and secret key sk .
 - 2: $(vk, sk) \leftarrow \text{SIG.KGen}(1^\lambda)$
 - 3: **return** (vk, sk)
-

2.2.2 AKE.Init

Executed by the Initiator $\mathcal{I}$ to begin the key exchange. It generates the ephemeral key material and a random nonce.

Algorithm 2 AKE.Init(1^λ)

- 1: **Output:** Initiator's message $\text{msg}_{I,1}$ and internal state $st_{\mathcal{I}}$.
 - 2: $s_I \leftarrow \{0, 1\}^{32}, N_I \leftarrow \{0, 1\}^{256}$ ▷ Create a 32-bit SPI and generate a 256-bit nonce
 - 3: $(ek, dk) \leftarrow \text{KEM.KGen}(1^\lambda)$ ▷ Generate ephemeral KEM keys
 - 4: $s' = s_I \parallel 0$ ▷ Half-open SPI
 - 5: $\text{msg}_{I,1} \leftarrow (s', ek, N_I)$
 - 6: $st_{\mathcal{I}} \leftarrow (\text{msg}_{I,1}, dk)$
 - 7: **return** $(\text{msg}_{I,1}, st_{\mathcal{I}})$
-

2.2.3 AKE.Respond

Executed by the Responder $\mathcal{R}$ upon receiving $\text{msg}_{I,1}$. It encapsulates the shared secret and generates a nonce.

Algorithm 3 AKE.Respond($\text{msg}_{I,1}$)

- 1: **Input:** Initiator's message $\text{msg}_{I,1}$.
 - 2: **Output:** Responder's message $\text{msg}_{R,1}$ and internal state $st_{\mathcal{R}}$.
 - 3: $s_R \leftarrow \{0, 1\}^{32}, N_R \leftarrow \{0, 1\}^{256}$ ▷ Create a 32-bit SPI and generate a 256-bit nonce
 - 4: $(s', ek, N_I) \leftarrow \text{msg}_{I,1}$ ▷ Parse incoming message
 - 5: $(K, ct_{kem}) \leftarrow \text{KEM.Encap}(ek)$ ▷ Encapsulate the KEM shared secret
 - 6: $s = s_I \parallel s_R$ ▷ Complete SPI
 - 7: $\text{msg}_{R,1} \leftarrow (s, ct_{kem}, N_R)$
 - 8: $st_{\mathcal{R}} \leftarrow (\text{msg}_{I,1}, \text{msg}_{R,1}, K)$
 - 9: **return** $(\text{msg}_{R,1}, st_{\mathcal{R}})$
-

2.2.4 AKE.AuthInit (Shared Secret Derivation & Failure Handling)

Executed by the Initiator $\mathcal{I}$ upon receiving $\text{msg}_{R,1}$. This algorithm handles the decapsulation and explicitly manages the decryption failure (Rigidity) to ensure IND-CPAF security. If successful, it derives the shared secret and generates the authentication token.

Algorithm 4 AKE.AuthInit($\text{sk}_I, ID_I, st_{\mathcal{I}}, \text{msg}_{R,1}$)

```
1: Input: Initiator's secret key  $\text{sk}_I$ , identity  $ID_I$ , state  $st_{\mathcal{I}}$ , and Responder's message  $\text{msg}_{R,1}$ .
2: Output: Initiator's authentication message  $\text{msg}_{I,2}$ , updated state  $st'_{\mathcal{I}}$  or failure symbol  $\perp$ .
3:  $(s', ek, N_I) \leftarrow st_{\mathcal{I}}; (s, ct_{kem}, N_R) \leftarrow \text{msg}_{R,1}$   $\triangleright$  Retrieve state and parse incoming message
4: Ensure  $s'(s_I) == s(s_I)$ 
5:  $K' \leftarrow \text{KEM.Decap}(dk, ct_{kem})$ 
6: if  $K' = \perp$  then
7:   return  $\perp$   $\triangleright$  Rigid decapsulation failure detected; primitive explicitly aborts
8: end if
9:  $(SS, mk_I, mk_R) \leftarrow \text{KDF}^+(K', N_I \parallel N_R)$   $\triangleright$  Derive the shared secret and MAC key
10:  $\text{DTBS}_I = \text{'0'} \parallel (s_I, s_R) \parallel ek \parallel ct_{kem}$ 
11:  $\sigma_I \leftarrow \text{SIG.Sign}(\text{sk}_I, \text{DTBS}_I)$   $\triangleright$  Sign SPI and key material
12:  $\tau_I \leftarrow \text{MAC}(mk_I, \text{'0'} \parallel (s_I, s_R) \parallel ID_I)$   $\triangleright$  Compute the MAC
13:  $\text{msg}_{I,2} \leftarrow (ID_I, \sigma_I, \tau_I)$ 
14:  $st'_{\mathcal{I}} \leftarrow (\text{msg}_{R,1}, SS, mk_R)$ 
15: return  $(\text{msg}_{I,2}, st'_{\mathcal{I}})$ 
```

2.2.5 AKE.AuthResp

Executed by the Responder $\mathcal{R}$ to verify the Initiator's identity and generate its own authentication token.

Algorithm 5 AKE.AuthResp($\text{sk}_R, ID_R, st_{\mathcal{R}}, \text{msg}_{I,2}, \text{vk}_I$)

```
1: Input: Responder's secret key  $\text{sk}_R$ , identity  $ID_R$ , state  $st_{\mathcal{R}}$ , Initiator's message  $\text{msg}_{I,2}$ , and Initiator's public key  $\text{vk}_I$ .
2: Output: Responder's authentication message  $\text{msg}_{R,2}$  and derived shared secret  $SS$ , or  $\perp$ .
3:  $(\text{msg}_{I,1}, \text{msg}_{R,1}, K) \leftarrow st_{\mathcal{R}}; (ID_I, \sigma_I, \tau_I) \leftarrow \text{msg}_{I,2}$   $\triangleright$  Retrieve state and parse incoming message
4:  $(\cdot, ek, N_I) \leftarrow \text{msg}_{I,1}$ 
5:  $(s, ct_{kem}, N_R) \leftarrow \text{msg}_{R,1}$ 
6:  $(SS, mk_I, mk_R) \leftarrow \text{KDF}^+(K, N_I \parallel N_R)$ 
7:  $\text{DTBS}_I = \text{'0'} \parallel (s_I, s_R) \parallel ek \parallel ct_{kem}$ 
8: if  $\text{SIG.Verify}(\text{vk}_I, \text{DTBS}_I, \sigma_I) \neq 1$  or  $\text{MAC}(mk_I, \text{'0'} \parallel (s_I, s_R) \parallel ID_I) \neq \tau_I$  then
9:   return  $\perp$ 
10: end if
11:  $\text{DTBS}_R = \text{'1'} \parallel (s_I, s_R) \parallel ct_{kem} \parallel ek$ 
12:  $\sigma_R \leftarrow \text{SIG.Sign}(\text{sk}_R, \text{DTBS}_R)$   $\triangleright$  Sign SPI and key material
13:  $\tau_R \leftarrow \text{MAC}(mk_R, \text{'1'} \parallel (s_I, s_R) \parallel ID_R)$ 
14:  $\text{msg}_{R,2} \leftarrow (ID_R, \sigma_R, \tau_R)$ 
15: return  $(\text{msg}_{R,2}, SS)$ 
```

2.2.6 AKE.Confirm

Executed by the Initiator $\mathcal{I}$ to finalize the exchange by verifying the Responder's authentication token.

Algorithm 6 AKE.Confirm($st'_T, \text{msg}_{R,2}, \text{vk}_R$)

- 1: **Input:** Initiator's updated state st'_T , Responder's message $\text{msg}_{R,2}$, and Responder's public key vk_R .
 - 2: **Output:** derived shared secret SS (accept), or $\perp$ (reject).
 - 3: $(\text{msg}_{R,1}, SS, mk_R) \leftarrow st'_T$; $(ID_R, \sigma_R, \tau_R) \leftarrow \text{msg}_{R,2}$ ▷ Retrieve state and parse incoming message
 - 4: $(s, ct_{kem}, N_R) \leftarrow \text{msg}_{R,1}$
 - 5: $\text{DTBS}_R = \text{'1'} \parallel (s_I, s_R) \parallel ct_{kem} \parallel ek$
 - 6: **if** SIG.Verify($\text{vk}_R, \text{DTBS}_R, \sigma_R$) $\neq 1$ **or** MAC($mk_R, \text{'1'} \parallel (s_I, s_R) \parallel ID_R$) $\neq \tau_R$ **then**
 - 7: **return** $\perp$
 - 8: **end if**
 - 9: **return** SS
-

2.2.7 KEM components

The KEM components below use three length parameters, all fixed per security level in Section 2.3: the shared-secret length $\text{len}_{ss} = |\mu|/8$ bytes (so m, r each have $8\text{len}_{ss} = |\mu|$ bits), the rigidity-tag length len_F (the byte length of $t = F_m(c)$), and the public-matrix seed length len_{seed} .

Algorithm 7 LOOM.CPAFKEM.KGen(1^λ): KEM Key Generation

Ensure: KEM Secret key $dk \in \mathcal{B}^{12 \cdot k \cdot n/8}$

Ensure: KEM Public key $ek \in \mathcal{B}^{d_t \cdot k \cdot n/8 + \text{len}_{seed}}$

- 1: **return** $(ek, dk) = \text{LOOM.CPAPKE.KeyGen}(1^\lambda)$
-

Algorithm 8 LOOM.CPAFKEM.Encap(ek): KEM Encapsulation

Require: KEM Public key $ek \in \mathcal{B}^{d_t \cdot k \cdot n/8 + \text{len}_{seed}}$

Ensure: Ciphertext $ct \in \mathcal{B}^{(d_u \cdot k + d_v)n/8 + \text{len}_F}$

Ensure: Shared secret $K \in \mathcal{B}^{\text{len}_{ss}}$

- 1: $m, r \leftarrow \$_{ \{0, 1\}^{8 \cdot \text{len}_{ss}} }$ ▷ Sample random coins
 - 2: $c = \text{LOOM.CPAPKE.Enc}(ek, m; r)$ ▷ Encrypt using derived coins
 - 3: $t = F_m(c)$ ▷ use m as PRF key
 - 4: **return** $(ct = (c \parallel t), K = G(m))$ ▷ Output ciphertext and derive shared secret
-

Algorithm 9 LOOM.CPAFKEM.Decap(dk, ct): KEM Decapsulation

Require: KEM Secret key $dk = \text{sk}_{pke}$

Require: Ciphertext $ct = (c \parallel t) \in \mathcal{B}^{(d_u \cdot k + d_v)n/8 + \text{len}_F}$

Ensure: Shared secret $K \in \mathcal{B}^{\text{len}_{ss}}$

- 1: $m' = \text{LOOM.CPAPKE.Dec}(dk, c)$ ▷ Decrypt the ciphertext
 - 2: **if** $m' = \perp$ **or** $F_{m'}(c) \neq t$ **then**
 - 3: **return** $\perp$ ▷ Invalid ciphertext: explicit rejection by throwing an exception.
 - 4: **end if**
 - 5: **return** $K = G(m')$ ▷ Success: derive shared secret
-

Algorithm 10 LOOM.CPAPKE.KeyGen(1^λ): Key Pair Generation

Ensure: Secret key $sk = \hat{s} \in \mathcal{R}_q^k \rightarrow \mathcal{B}^{12 \cdot k \cdot n/8}$

Ensure: Public key $pk = (t' \in \mathcal{R}_{2^{d_t}}^k, seed_A) \rightarrow \mathcal{B}^{d_t \cdot k \cdot n/8 + \text{len}_{seed}}$

```
1:  $seed_0 \leftarrow \mathcal{B}^{\text{len}_{seed}}$ 
2:  $(seed_A, seed_{sk}) = \text{XOF}(seed_0)$   $\triangleright seed_A, seed_{sk} \in \mathcal{B}^{\text{len}_{seed}}$ 
3: for  $i$  from 0 to  $k - 1$  do
4:   for  $j$  from 0 to  $k - 1$  do
5:      $\hat{A}[i][j] \leftarrow \text{XOF}_{\text{pub}}(seed_A, i, j)$   $\triangleright$  Generate matrix  $\hat{A} \in \mathcal{R}_q^{k \times k}$  in NTT domain
6:   end for
7: end for
8:  $ctr = 0$ 
9: for  $i$  from 0 to  $k - 1$  do
10:   $s[i] \leftarrow \text{CBD}_{\eta_1}(\text{PRF}(seed_{sk}, ctr))$   $\triangleright$  Sample  $s \in \mathcal{R}_q^k$ 
11:   $ctr = ctr + 1$ 
12: end for
13:  $\hat{s} = \text{NTT}(s)$ 
14:  $t = \text{NTT}^{-1}(\hat{A} \circ \hat{s})$   $\triangleright$  Pure MLWR: no explicit error  $e$  is added
15:  $sk = \text{Encode}_{12}(\hat{s})$ 
16:  $pk = (\text{Encode}_{d_t}(\text{Compress}_q(t, d_t)), seed_A)$ 
17: return  $(pk, sk)$ 
```

Algorithm 11 LOOM.CPAPKE.Enc($pk, \mu; seed_1$): Encryption

Require: Public key $pk \in \mathcal{B}^{d_t \cdot k \cdot n/8 + \text{len}_{seed}}$

Require: Message $\mu \in \mathcal{B}^{\text{len}_{ss}}$

Require: Random coins $seed_1 \in \mathcal{B}^{\text{len}_{seed}}$

Ensure: Ciphertext $ct = (c_1, c_2) \in ((\mathbb{Z}_{2^{d_u}}^n)^k, \mathbb{Z}_{2^{d_v}}^n) \rightarrow \mathcal{B}^{(d_u \cdot k + d_v)n/8}$

```
1:  $t' \leftarrow \text{Decode}_{d_t}(pk[0 : d_t \cdot k \cdot n/8])$   $\triangleright$  Decode the compressed public-key vector
2:  $seed_A \leftarrow pk[d_t \cdot k \cdot n/8 : d_t \cdot k \cdot n/8 + \text{len}_{seed}]$   $\triangleright$  Extract the matrix seed
3:  $ctr = 0$ 
4:  $t_{\text{lift}} = \text{Inv}(t', d_t; \text{PRF}(seed_1, ctr))$   $\triangleright$  Derandomized lifting using  $seed_1$ 
5:  $ctr = ctr + 1$ 
6:  $\hat{t} = \text{NTT}(t_{\text{lift}})$ 
7: for  $i$  from 0 to  $k - 1$  do
8:   for  $j$  from 0 to  $k - 1$  do
9:      $\hat{A}^T[i][j] \leftarrow \text{XOF}_{\text{pub}}(seed_A, j, i)$   $\triangleright$  Generate the transpose of matrix  $\hat{A}$ 
10:   end for
11: end for
12: for  $i$  from 0 to  $k - 1$  do
13:   $r[i] \leftarrow \text{CBD}_{\eta_2}(\text{PRF}(seed_1, ctr))$   $\triangleright$  Sample  $r \in \mathcal{R}_q^k$ 
14:   $ctr = ctr + 1$ 
15: end for
16:  $\hat{r} = \text{NTT}(r)$ 
17:  $w = \text{MsgEncode}(\mu)$   $\triangleright w \in \mathbb{Z}_q^n$ 
18:  $u = \text{NTT}^{-1}(\hat{A}^T \circ \hat{r})$   $\triangleright$  No explicit error  $e'$ 
19:  $v_0 = \text{NTT}^{-1}(\hat{t}^T \circ \hat{r})$   $\triangleright$  No explicit error  $e''$ 
20:  $c_1 = \text{Encode}_{d_u}(\text{Compress}_q(u, d_u))$ 
21:  $c_2 = \text{Encode}_{d_v}(\text{Compress}_q(v_0 + w, d_v))$   $\triangleright$  Add message in  $\mathcal{R}_q$  before compression
22: return  $ct = (c_1 \parallel c_2)$ 
```

Algorithm 12 $\text{Inv}(\mathbf{t}', d_t; \text{seed})$: Randomized Lifting

Require: Compressed public key $\mathbf{t}' \in \mathcal{R}_{2^{d_t}}^k$ **Require:** Decompress parameter d_t **Require:** Random coins $\text{seed} \in \mathcal{B}^{\text{len}_{\text{seed}}}$ **Ensure:** Re-randomized decompressed public key $\mathbf{t}_{\text{lift}} \in \mathcal{R}_q^k$

```
1:  $\rho = \text{PRF}(\text{seed})$  ▷ Deterministic pseudorandom byte stream
2: for  $i$  from 0 to  $k - 1$  do
3:   for  $j$  from 0 to  $n - 1$  do
4:      $B = \text{Preim}_q^\pm(\mathbf{t}'^{(i)}[j], d_t)$  ▷ Error bucket for this coefficient
5:      $\iota = \text{SampleIndex}(\rho, i \cdot n + j, |B|)$  ▷ Index in  $\{0, \dots, |B| - 1\}$  from  $\rho$ 
6:      $e = B[\iota]$  ▷ Deterministic selection from the bucket
7:      $\mathbf{t}_{\text{lift}}^{(i)}[j] = \text{Decompress}_q(\mathbf{t}'^{(i)}[j], d_t) + e$ 
8:   end for
9: end for
10: return  $\mathbf{t}_{\text{lift}}$ 
```

Algorithm 13 $\text{LOOM.CPAPKE.Dec}(sk, ct)$: Decryption

Require: Secret key $sk \in \mathcal{B}^{12 \cdot k \cdot n/8}$ **Require:** Ciphertext $ct \in \mathcal{B}^{(d_u \cdot k + d_v) \cdot n/8}$ **Ensure:** Message $\mu \in \mathcal{B}^{\text{len}_{\text{ss}}}$

```
1:  $c_1 \leftarrow ct[0 : d_u \cdot k \cdot n/8]$  ▷ Extract the encoded first ciphertext component
2:  $c_2 \leftarrow ct[d_u \cdot k \cdot n/8 : (d_u \cdot k + d_v) \cdot n/8]$  ▷ Extract the encoded second ciphertext component
3:  $\mathbf{u} = \text{Decompress}_q(\text{Decode}_{d_u}(c_1), d_u)$  ▷ Deterministic canonical decomposition
4:  $\mathbf{v} = \text{Decompress}_q(\text{Decode}_{d_v}(c_2), d_v)$ 
5:  $\hat{\mathbf{s}} = \text{Decode}_{12}(sk)$ 
6:  $\mathbf{w}' = \mathbf{v} - \text{NTT}^{-1}(\hat{\mathbf{s}}^T \circ \text{NTT}(\mathbf{u})) \bmod^+ q$ 
7:  $\mu = \text{MsgDecode}(\mathbf{w}')$ 
8: return  $\mu$ 
```

Algorithm 14 $\text{MsgEncode}(\mu)$: Message Encoding

Require: Message bits $\mu \in \mathcal{B}^{\text{len}_{\text{ss}}}$ **Ensure:** Encoded element $w \in \mathbb{Z}_q^n$

```
1: for  $i$  from 0 to  $n - 1$  do
2:    $w_i = \frac{q-1}{2} \cdot \mu$ 
3: end for
4: return  $w$ 
```

Algorithm 15 $\text{MsgDecode}(w)$: Message Decoding

Require: Decrypted ring element $w \in \mathcal{R}_q$ **Ensure:** Decoded Message bits $\mu \in \mathcal{B}^{\text{len}_{\text{ss}}}$

```
1: for  $i$  from 0 to  $n - 1$  do
2:    $\mu' = \left\lceil \frac{2}{q} \cdot \bar{w}_i \right\rceil \bmod 2$ 
3: end for
4: return  $\mu'$ 
```

2.2.8 Signature Component

LOOM-AKE utilizes a digital signature scheme as an abstract, black-box authentication component, denoted by $\text{SIG} = (\text{SGen}, \text{Sign}, \text{Verify})$. The LOOM-AKE protocol state machine and its security reduction treat signatures as opaque byte strings, thereby decoupling the key exchange logic from the internal algebraic complexities of the underlying signature scheme.

In the concrete NGCC submission suite, the abstract scheme SIG is normatively instantiated by **Shuttle**, a lattice-based digital signature scheme submitted in parallel to this algorithm call. The complete specification of Shuttle—including its key generation, signing, verification, asymmetric stretch-and-compress mechanism, rejection-free iterative sampler, and rANS entropy coding—is detailed in its official algorithm proposal [16].

Interface and Key Mapping. The abstract interfaces of SIG used in AKE.KeyGen , AKE.AuthInit , AKE.AuthResp , and AKE.Confirm are mapped directly to Shuttle’s native algorithms as follows:

- $\text{SIG.SGen}(1^\lambda) \equiv \text{Shuttle.KeyGen}(\xi)$: Generates the static verification key $\text{vk} \leftarrow pk_{\text{shuttle}}$ and the static signing key $\text{sk} \leftarrow sk_{\text{shuttle}}$ from a random seed $\xi \in \{0, 1\}^\lambda$.
- $\text{SIG.Sign}(\text{sk}, M) \equiv \text{Shuttle.Sign}(sk_{\text{shuttle}}, M, \text{rnd})$: Generates a signature $\sigma \leftarrow (\text{seed}_c, z_1, h)$ on the message M using the signing key sk_{shuttle} and optional randomness rnd . The output σ is formatted as a canonical byte string via Shuttle’s `sigEncode` routine.
- $\text{SIG.Verify}(\text{vk}, M, \sigma) \equiv \text{Shuttle.Verify}(pk_{\text{shuttle}}, M, \sigma)$: Verifies the signature σ against the message M and verification key pk_{shuttle} , returning 1 if accepted, and 0 otherwise.

Parameter Alignment and Suite Pairing. To satisfy the NGCC requirement of supporting 128-bit, 256-bit, and 512-bit classical security levels, the instances of Loom-KEM and Shuttle are paired as shown in Table 2. This pairing ensures that the security margin of the authentication channel (SIG) is always commensurate with that of the key exchange channel (KEM).

Table 2: Parameter Pairing and Handshake Bandwidth for Loom Suite

Instance	Security	KEM Component	SIG Component	Total Bandwidth (Bytes) [†]
Loom-128	128-bit	Loom-KEM-128	SHUTTLE-128	3870
Loom-256	256-bit	Loom-KEM-256	SHUTTLE-256	7522
Loom-512	512-bit	Loom-KEM-512	SHUTTLE-512	15986

[†] *Total Bandwidth* is the sum of all cryptographic payloads transmitted in a complete mutual authentication handshake, calculated as $|ek| + |ct_{\text{kem}}| + 2|\sigma_{\text{SIG}}|$.

Security Decoupling. The security proof of LOOM-AKE (Theorem 4.13) relies solely on the existential unforgeability under chosen-message attacks (UF-CMA) of the signature scheme SIG . The internal design choices of Shuttle—such as the elimination of the long-tail rejection bottleneck via the transition step $T_v(y)$ and the logarithmic approximation of transcendental functions—do not affect the security reduction of LOOM, provided that Shuttle’s output distribution remains statistically close to the target discrete Gaussian distribution as proven in the security analysis of Shuttle [16].

2.3 Symmetric Primitives and Quantum Security Levels

LOOM-AKE relies on several symmetric components at both the AKE layer and the KEM layer. The hash and XOF components are modeled as random oracles or ideal extendable-output functions where appropriate in Section 4, while the keyed components are assumed to

Algorithm 16 $\text{KDF}^+(K, \text{salt})$: key derivation

Require: KEM shared secret K

Require: Sult $\text{salt} = N_I \parallel N_R$

Ensure: Three derived keys in the order of SS , mk_I and mk_R .

- 1: $SS = \text{KDF}(K, \text{salt} \parallel 0\text{x}01)$
 - 2: $mk_I = \text{KDF}(K, SS \parallel \text{salt} \parallel 0\text{x}02)$
 - 3: $mk_R = \text{KDF}(K, mk_I \parallel \text{salt} \parallel 0\text{x}03)$
-

satisfy the corresponding PRF or MAC security notions. More specifically, the AKE layer uses the key-derivation function KDF to derive the session key SS and the authentication keys mk_I, mk_R , and uses the message authentication code MAC for identity binding. The KEM layer uses the extendable-output functions $\text{XOF}_{\text{pub}}, \text{XOF}$, the pseudorandom function PRF, the hash function G for KEM shared-secret derivation, and the keyed tag function F for rigidity checking.

Message authentication code. The message authentication code MAC provides identity binding in the authentication phase. Concretely, the tags contained in the authentication messages are of the form

$$\tau_I = \text{MAC}(mk_I, \text{macmsg}_I), \quad \tau_R = \text{MAC}(mk_R, \text{macmsg}_R),$$

where the MAC messages are bound to the session transcript. The required security notion for MAC is EUF-CMA security under session-specific keys. The MAC keys are derived by KDF^+ using role-separated labels, and the tag lengths are specified in Table 3. For the 128-bit transition-oriented parameter set, the 128-bit MAC tag length is aligned with existing 128-bit symmetric primitives and with the 128-bit session-key interface. In an online AKE protocol, MAC forgery attempts are also constrained by the number of protocol sessions and verification attempts, rather than providing an unrestricted offline verification oracle.

AKE key derivation. The function KDF derives the final session key and the two MAC keys from the KEM shared secret K and the nonce-derived salt. The proof of LOOM-AKE relies on the PRF security of this derivation, not merely on collision resistance of an underlying hash function. Therefore, KDF^+ uses explicit domain separation for the three outputs:

$$\begin{aligned} SS &= \text{KDF}(K, \text{salt} \parallel 0\text{x}01), \\ mk_I &= \text{KDF}(K, \text{salt} \parallel 0\text{x}02), \quad mk_R = \text{KDF}(K, \text{salt} \parallel 0\text{x}03). \end{aligned}$$

Equivalently, one may instantiate KDF^+ by a standard extract-and-expand KDF and parse the output into $SS \parallel mk_I \parallel mk_R$, provided that the three derived keys are domain separated and pseudorandom.

Secret-bearing derivation and sampling. The secret-bearing functions XOF and PRF are used in the KEM to expand secret seeds into internal secret material. For example, XOF splits a master seed via

$$(\text{seed}_A, \text{seed}_{sk}) = \text{XOF}(\text{seed}_0),$$

where seed_{sk} is secret, and PRF expands secret seeds into the secret vectors $\mathbf{s}, \mathbf{r}$ and drives randomized lifting. These components must provide pseudorandom expansion from secret seeds. When recovering the secret seed from public information is the relevant generic attack, the corresponding security level is governed by preimage or key-search complexity.

Public pseudorandom expansion. The function XOF_{pub} expands the public seed seed_A into the public matrix $\hat{\mathbf{A}}$ in `LOOM.CPAPKE.KeyGen` and `LOOM.CPAPKE.Enc`. Both its input and output are public, and no secret ever passes through it. Consequently, XOF_{pub} is not required to provide preimage resistance or collision resistance for secret protection. Its requirement is distributional: the induced distribution of $\hat{\mathbf{A}}$ must be statistically or computationally close to the specified uniform distribution over the public matrix space, including the negligible statistical distance introduced by rejection sampling. This requirement is independent of the secret-key security level, except that the distributional error is chosen negligible at the target security level.

KEM message and shared-secret derivation. The KEM encapsulation encrypts an internal message

$$m \in \{0, 1\}^{|\mu|}$$

under the underlying PKE. The length $|\mu|$ is fixed by the PKE plaintext space and is set to 128, 256, and 512 bits for the three parameter sets, respectively. The hash function $\mathbf{G}$ is used to derive the KEM shared secret

$$K = \mathbf{G}(m).$$

For the AKE proof, the resulting K is used as the keying material for the KDF/PRF layer. Since K is derived from m , the entropy source for the KEM shared secret is the encapsulated message m . Therefore, the KEM encapsulated-message length, the output length of $\mathbf{G}$, the KEM shared-secret length, and the final AKE session-key length are all set to 128, 256, and 512 bits for the three parameter sets, respectively, as specified in Table 3.

Rationale for the 128-bit encapsulated message. For the 128-bit classical security level, LOOM-AKE derives the KEM shared secret from a 128-bit encapsulated message. This choice is intentional and follows the transition-oriented security requirements of the New Generation Commercial Cryptography (NGCC) algorithm call: the 128-bit public-key level is primarily intended to interoperate with existing 128-bit symmetric primitives during the migration period. In particular, a 128-bit encapsulated message naturally matches the security strength, key sizes, and design paradigms of widely deployed 128-bit symmetric primitives, such as SM4 and conventional 128-bit session keys.

We stress that the statement “a 128-bit encapsulated message” should not be interpreted as giving a complete quantum attack of cost 2^{64} elementary operations. A generic Grover search over a 128-bit message space has query complexity about 2^{64} , but these are oracle-query counts. Each Grover iteration must implement a coherent verification oracle for testing whether a candidate message is consistent with the observed ciphertext. For an FO-style KEM, this verification oracle must at least recompute the deterministic encapsulation or re-encryption procedure and compare the resulting ciphertext with the target ciphertext.

This distinction is analogous to the standard interpretation of AES-128 key search in the NIST PQC security categories [43]. Although Grover search over a 128-bit AES key uses about 2^{64} oracle queries, each AES oracle evaluation has non-negligible quantum circuit complexity. The commonly used reference interpretation assigns a substantial elementary-gate cost to one AES-128 oracle evaluation, leading to an overall gate-level cost around the 2^{80} scale for AES-128 key search [25, 35].

The verification oracle for LOOM is no simpler than a single AES-128 evaluation. Even ignoring the cost of hash, XOF, PRF, and KDF components, one deterministic KEM re-encryption involves the algebraic core of the underlying module-lattice PKE, including sampling from the prescribed noise distribution, public-matrix expansion or access, NTT-domain module arithmetic, polynomial multiplications over R_q , coefficient-wise modular arithmetic, compression, and ciphertext comparison. For the 128-bit parameter set, the core computation contains a

module-vector product and an inner product, requiring multiple polynomial multiplications together with forward and inverse NTTs. Therefore, the coherent quantum circuit implementing one LOOM verification oracle is conservatively no cheaper than an AES-128 oracle and is expected to be substantially more expensive in practice.

Consequently, the raw 2^{64} Grover query count for a 128-bit encapsulated message should not be read as a 2^{64} quantum gate-cost attack on the KEM or on the AKE session key. At the gate-complexity level, a purely generic Grover attack against the 128-bit encapsulated message is at least comparable to the 2^{80} reference cost associated with AES-128 key search, and realistically larger for the lattice-based verification oracle considered here. This generic-search argument is independent of the concrete lattice-attack estimates for the underlying PKE, which provide the primary basis for the claimed post-quantum security of the KEM. Hence, the use of a 128-bit encapsulated message at the 128-bit classical level is consistent with the NGCC transition rationale and with the intended 80-bit quantum security reference level.

Rigidity tag function. The function F is used to compute the rigidity tag

$$t = F_m(c),$$

which is recomputed during decapsulation to detect invalid or inconsistent ciphertexts. Its role is stronger than ordinary pseudorandom expansion: the tag must bind the secret value and the ciphertext in such a way that an adversary cannot produce a ciphertext that passes verification but decapsulates to an incorrect non- $\perp$ key. We therefore assign F a collision-resistance-style output length. This conservative choice accounts for both classical birthday attacks and quantum collision-search attacks.

Security-level conventions. Let (λ_C, λ_Q) denote the targeted classical and quantum security levels. We distinguish two classes of generic attacks on the symmetric components.

For preimage, key-search, and PRF-distinguishing attacks over a b -bit secret or output space, Grover search gives a generic quantum query cost of about $2^{b/2}$. Therefore, for secret seeds and secret-bearing XOF or PRF outputs, we require

$$b_{\text{prf}} \geq \max(\lambda_C, 2\lambda_Q).$$

The actual lengths are rounded up to the standard values shown in Table 3. The KEM encapsulated-message length $|\mu|$ is listed separately, because it is fixed by the plaintext space of the underlying PKE and is justified for the 128-bit transition-oriented parameter set by the gate-complexity rationale above.

For components whose security relies on collision resistance, an n -bit output gives approximately $2^{n/2}$ classical collision security and approximately $2^{n/3}$ quantum collision security against BHT-type collision attacks. Therefore, for collision-resistance-style uses, we require

$$b_{\text{cr}} \geq \max(2\lambda_C, 3\lambda_Q).$$

The rigidity tag function F is assigned the collision-resistance parameter b_{cr} , whereas XOF and PRF are assigned the secret-bearing PRF/preimage parameter b_{prf} . The public expansion function XOF_{pub} is instead governed by its distributional uniformity requirement.

The nonces N_I, N_R are public freshness values used to prevent replay and cross-session collisions. We use 256-bit nonces for the 128-bit and 256-bit parameter sets and 512-bit nonces for the 512-bit parameter set, which makes nonce collision and replay-related bad events negligible even across a very large number of sessions. Larger nonces may be used without affecting the security proof.

We do not mandate concrete instantiations in the normative specification. Instead, Table 3 specifies the required output, key, seed, message, and tag lengths. Any primitive named in

a reference implementation is an example instantiation and may be replaced by a stronger primitive with the same interface. In particular, the LOOM-AKE-512 suite is intended for the NGCC setting and targets 512-bit classical security at the symmetric layer; its symmetric primitives must therefore provide the corresponding 512-bit PRF/preimage strength and 1024-bit collision-resistance-style tag length where specified.

Table 3: Security parameters (in bits) for the symmetric components of LOOM-AKE.

Parameter	LOOM-AKE-128	LOOM-AKE-256	LOOM-AKE-512
Target classical strength λ_C	128	256	512
Target quantum strength λ_Q	80	128	256
$b_{\text{prf}} \geq \max(\lambda_C, 2\lambda_Q)$	256	256	512
$b_{\text{cr}} \geq \max(2\lambda_C, 3\lambda_Q)$	256	512	1024
KEM encapsulated-message length $ \mu $	128	256	512
G output / KEM shared-secret length $ K $	128	256	512
Session-key length $ SS $	128	256	512
KDF output length for mk_I, mk_R	128	256	512
MAC key/tag length $ mk = \tau $	128	256	512
XOF, PRF secret-bearing output/key length	256	256	512
F rigidity-tag output length	256	512	1024
XOF _{pub} requirement	public pseudorandom expansion / distributional uniformity		
Seed length $8\text{len}_{\text{seed}}$	256	256	512
Nonce length $ N_I = N_R $	256	256	512

2.4 Parameter Sets and Suite Composition

This section specifies the concrete LOOM-AKE suites and the parameters of their underlying cryptographic components. Each AKE suite is defined by the combination of three components: a Loom-KEM parameter set, a Shuttle signature parameter set, and the symmetric authentication parameters (such as the MAC tag length) used at the AKE layer. The three suites are designed to target the 128-bit, 256-bit, and 512-bit classical security levels, aligning with the migration requirements of the target security categories. In particular, the LOOM-AKE-512 suite is tailored for the NGCC setting, matching the highest classical and post-quantum security requirements.

Table 4 summarizes the suite composition and the payload sizes of the individual cryptographic primitives. The KEM columns report the ephemeral public key size $|ek|$ and ciphertext size $|ct|$ of the selected Loom-KEM instance. The signature column reports the signature size $|\sigma|$ of the corresponding Shuttle instance used for transcript signing. The MAC column reports the tag length $|\tau|$ used for identity binding in the AKE authentication messages. The reported decryption failure rates (DFR) are associated with the underlying Loom-KEM instances, which are handled explicitly by the AKE state machine (i.e., a failed decapsulation triggers an immediate session abort and state erasure).

Table 4: LOOM-AKE suite composition and primitive payload sizes (in bytes).

AKE Suite	Shuttle	Loom-KEM	Sig. $ \sigma $	MAC $ \tau $	KEM ($ ek , ct $)
AKE-128	Level-128	KEM-128	1183	16	(752, 736)
AKE-256	Level-256	KEM-256	2417	32	(1312, 1344)
AKE-512	Level-512	KEM-512	5001	64	(2880, 2944)

Table 5 lists the core algebraic parameters of the underlying Loom-KEM instances. The

parameter $|\mu|$ denotes the bit-length of the internal plaintext message encrypted by the underlying PKE, which serves as the entropy source for the derived KEM shared secret $K = \mathsf{G}(m)$. The parameters n, k, q define the module rank and ring modulus, η_1, η_2 specify the centered-binomial noise distributions, and (d_t, d_u, d_v) define the compression widths for the public key and ciphertext components. By trading a minor decryption failure rate (DFR) for aggressive compression, Loom-KEM achieves highly compact public key and ciphertext sizes while maintaining the concrete security margins required for each level.

Table 5: Core Loom-KEM parameter sets used by the LOOM-AKE suites.

KEM Instance	Level	$ \mu $	n	k	q	η_1	η_2	(d_t, d_u, d_v)	DFR
LOOM-KEM-128	1	128	128	5	3329	3	2	(9, 8, 4)	$2^{-19.7}$
LOOM-KEM-256	3	256	256	4	7681	8	4	(10, 9, 4)	$2^{-18.9}$
LOOM-KEM-512	5	512	512	4	7681	10	8	(11, 10, 4)	$2^{-24.8}$

3 Design Rationale

3.1 Underlying Hard Problems

The security of the proposed LOOM-AKE primitive relies on the well-established hardness of lattice problems over polynomial rings, specifically the Shifted Module Learning with Rounding (Shifted-MLWR) problem for confidentiality, and the Module Short Integer Solution (MSIS) problem for authentication.

Shifted Module Learning with Rounding (Shifted-MLWR): The confidentiality of the ephemeral key exchange in LOOM-AKE is based on the Shifted-MLWR problem, instantiated via our dedicated LOOM-KEM. Let $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ be a polynomial ring. In standard MLWR, given a random matrix $\mathbf{A} \leftarrow R_q^{k \times k}$ and a secret vector $\mathbf{s} \leftarrow \chi^k$, the adversary’s task is to distinguish the rounded public key $(\mathbf{A}, \lfloor \frac{2^d}{q} \cdot \mathbf{A}\mathbf{s} \rfloor)$ from uniform. In LOOM-KEM, during the encryption phase, a public message-dependent shift $\Delta \in R_q^k$ (derived from the message embedding w) is injected prior to rounding. The Shifted-MLWR problem asserts that the distribution of:

$$\left(\mathbf{A}, \Delta, \left\lfloor \frac{2^d}{q} \cdot (\mathbf{A}\mathbf{s} + \Delta) \right\rfloor \right)$$

is computationally indistinguishable from the uniform distribution over $R_q^{k \times k} \times R_q^k \times R_{2^d}^k$. As established in the core theoretical framework of Weaver [17], there exists a rigorous security reduction from standard MLWE to Shifted-MLWR. This reduction proves that the public shift Δ does not alter the geometric invariants of the underlying lattice, nor does it affect the uniformity of the noise before rounding. This provides a solid theoretical foundation for LOOM-KEM to embed messages directly into the rounding process without explicitly adding error vectors.

Module Short Integer Solution (MSIS): The unforgeability of the underlying signature scheme (instantiated by Shuttle [16]) is based on the MSIS problem. Given a uniformly random matrix $\mathbf{A} \leftarrow R_q^{k \times l}$, the MSIS problem asks to find a non-zero vector $\mathbf{z} \in R_q^l$ such that $\mathbf{A}\mathbf{z} = \mathbf{0} \pmod{q}$ and $\|\mathbf{z}\|_2 \leq \beta$ (under the Euclidean norm) for a small bound β .

3.2 Parameter Selection Strategy

The parameter selection for LOOM-AKE is driven by the packet-size constraints of UDP-based key exchange protocols. In standard IKEv2, the initial key exchange materials are carried in the `IKE_SA_INIT` exchange, which is transmitted before any shared keys have been established.

Consequently, this initial exchange cannot use the application-layer IKEv2 fragmentation mechanism of RFC 7383 [44]. Oversized first-flight packets must either rely on IP-layer fragmentation, be split using `IKE.INTERMEDIATE` as specified in RFC 9242 [45], or be avoided entirely by selecting sufficiently compact key exchange parameters.

MTU Budget for Existing IKEv2. Let M denote the target path MTU. For IPv6 over UDP, the network-layer overhead is 48 bytes, consisting of a 40-byte IPv6 header and an 8-byte UDP header. Thus, the available UDP payload budget is

$$B_{\text{UDP}}(M) = M - 48.$$

For the IPv6 minimum MTU $M = 1280$, this yields $B_{\text{UDP}}(1280) = 1232$ bytes. For the standard Ethernet/cloud MTU $M = 1500$, this yields $B_{\text{UDP}}(1500) = 1452$ bytes.

In an unmodified IKEv2 implementation, the KEM material must share this budget with the standard `IKE.SA_INIT` payloads. Our packet captures show non-KEM overheads of 208 bytes in the initiator-to-responder direction and 241 bytes in the responder-to-initiator direction. Hence, the conservative KEM payload budgets are

$$B_{\text{KEM}}^{1280} = 1280 - 48 - 241 = 991 \text{ bytes},$$

and

$$B_{\text{KEM}}^{1500} = 1500 - 48 - 241 = 1211 \text{ bytes}.$$

We use the responder direction as the limiting case because it has the larger measured IKEv2 overhead.

Under these bounds, LOOM-AKE-128 fits within existing IKEv2 without modifying the protocol flow. Its KEM public key is $|ek| = 752$ bytes and ciphertext is $|ct| = 736$ bytes, giving a worst-case packet size of

$$48 + 241 + 752 = 1041 \text{ bytes}$$

for the initiator flight, and

$$48 + 241 + 736 = 1025 \text{ bytes}$$

for the responder flight. Both are comfortably below the IPv6 minimum MTU of 1280 bytes. Thus, the 128-bit instance supports fragmentation-free deployment in the standard `IKE.SA_INIT` exchange, without relying on IP-layer fragmentation or `IKE.INTERMEDIATE`.

MTU-Aware Deployment Profile for Larger Instances. The 256-bit instance has larger KEM payloads, namely $|ek| = 1312$ bytes and $|ct| = 1344$ bytes. These values exceed the payload budget of unmodified IKEv2, and therefore LOOM-AKE-256 is not intended as a wire-compatible drop-in replacement for the general-purpose `IKE.SA_INIT` exchange. Instead, it targets an MTU-aware fixed-suite deployment profile that preserves the high-level SIGMA-style structure of IKEv2 while omitting general-purpose negotiation and notification payloads from the first two flights.

The abstract LOOM primitive specifies the first two messages as

$$I \rightarrow R : (s', ek, N_I),$$

$$R \rightarrow I : (s, ct, N_R),$$

where $s' = s_I \parallel 0$, $s = s_I \parallel s_R$, and N_I, N_R are nonces. Under our concrete parameterization, s' and s occupy $|SPI| = 8$ bytes each, and each nonce N_X occupies 32 bytes. Therefore, the non-KEM protocol overhead of the first two LOOM flights is exactly 40 bytes.

Under a raw fixed-suite wire profile over UDP, we analyze the transmission footprint of the larger second flight ($\text{msg}_{R,1}$, carrying $|ct| = 1344$ bytes) based on the serialized layout defined

in our implementation. When encapsulated in a standard IPv6/UDP packet, the network-layer overhead consists of a 40-byte IPv6 header and an 8-byte UDP header, totaling 48 bytes. The resulting on-the-wire packet size is

$$48 \text{ (IPv6+UDP)} + 40 \text{ (Loom-AKE non-KEM)} + 1344 \text{ (KEM } ct) = 1432 \text{ bytes.}$$

For IPv4/UDP deployments, the network-layer overhead is reduced to 28 bytes (20-byte IPv4 header and 8-byte UDP header), yielding an on-the-wire size of

$$28 \text{ (IPv4+UDP)} + 40 \text{ (Loom-AKE non-KEM)} + 1344 \text{ (KEM } ct) = 1412 \text{ bytes.}$$

Both configurations fall comfortably below the standard 1500-byte Ethernet MTU. This leaves a sufficient budget (68 bytes under IPv6; 88 bytes under IPv4) for additional encapsulation headers, such as a minimalist VPN tunnel or custom framing headers, before triggering IP fragmentation. This analytical evaluation demonstrates that our choice of a 32-byte nonce ($|N_X| = 256$ bits) naturally aligns LOOM-AKE-256 with standard network MTU constraints without requiring any truncation of cryptographic entropy, ensuring that the first round-trip can be transmitted without IP-layer fragmentation.

Fragmentation After Key Establishment. The first two flights are required to remain unfragmented because no shared key is available before the KEM exchange completes. This is the same reason why RFC 7383 [44] cannot be applied to `IKE_SA_INIT`. After the responder ciphertext has been processed and the initiator successfully decapsulates, both parties hold the same KEM secret except with negligible rigidity error. A deployment protocol may then derive additional transcript-bound traffic keys from the LOOM shared secret and use them to protect the third and fourth authentication flights.

This enables an application-layer fragmentation mechanism analogous to RFC 7383 for large identity, certificate, or signature payloads (such as the Shuttle signatures in the second phase). Unlike unauthenticated fragmentation of the initial exchange, post-KEM fragments can be accepted only if they are associated with an existing handshake state and pass authentication under keys derived from the ephemeral KEM secret. This prevents an off-path adversary from forcing unbounded reassembly buffers for arbitrary forged fragments. Implementations should also enforce a small maximum fragment count and discard incomplete fragment sets after a short timeout.

In summary, the parameter strategy is two-tiered. The 128-bit instance is designed for direct integration into existing IKEv2 under the IPv6 minimum MTU of 1280 bytes. The 256-bit instance targets a fixed-suite MTU-aware deployment profile that preserves the SIGMA-style structure of IKEv2, keeps the first two KEM flights unfragmented under the standard 1500-byte MTU, and supports authenticated fragmentation only after KEM-derived keys are available.

3.3 Decryption Failure Rate (DFR) Analysis

This section mathematically and conceptually justifies our core design philosophy: safely trading perfect correctness for compact transmission sizes, while maintaining provable security through explicit failure detection. We argue that perfect correctness is overly restrictive for ephemeral key exchange protocols such as IKEv2. Instead of requiring a negligible decryption failure rate as in traditional IND-CCA-secure KEMs, LOOM-KEM deliberately permits a small but detectable DFR in order to enable more aggressive payload compression. Across our parameter sets, the resulting DFR ranges from 2^{-19} to 2^{-25} as listed in Table 4.

Mathematically, the decryption error polynomial E in LOOM-KEM is modeled as:

$$E = \mathbf{e}_1 \mathbf{s}_2 - \mathbf{e}_2 \mathbf{s}_1 + \mathbf{e}_3 + \mathbf{c}_u \mathbf{s}_2 + \mathbf{c}_v,$$

where $\mathbf{s}_1, \mathbf{s}_2$ are the secret vectors, $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$ are the inherent rounding errors from the MLWR rounding process, and $\mathbf{c}_u, \mathbf{c}_v$ are the additional errors introduced by aggressive bit-dropping compression parameters such as d_u and d_v . Decapsulation succeeds if and only if

$$\|E\|_\infty < \lfloor q/4 \rfloor.$$

By analyzing the convolution of these random variables under the selected compression parameters, we obtain the DFR estimates reported in Table 4. For example, a DFR of 2^{-22} corresponds to one decapsulation failure in about 2^{22} handshakes, i.e., approximately one failure per 4.2 million handshakes. Such rare failures can be handled by a simple bounded retry mechanism at the protocol or deployment layer.

Security Alignment via Explicit Failure Detection: A non-negligible DFR traditionally breaks IND-CCA-style reasoning because observable protocol aborts may act as a Failure Oracle, leaking information correlated with the decapsulation secret. We neutralize this threat and prove the security of LOOM-KEM under controlled DFR using two synergistic properties:

1. **Rigidity (Explicit Failure Detection):** It is cryptographically unsound to let decryption fail silently and rely on downstream MAC verification to catch the error. A silent failure may yield a biased or inconsistent key K' , which could then be propagated into the protocol layer. To prevent this, LOOM-KEM enforces *rigidity* by embedding a verification tag $t = F_m(c)$ in the ciphertext. The decapsulator deterministically detects decryption failures: if the verification tag does not match, it immediately aborts and outputs an explicit failure symbol $\perp$ (Algorithm 9). This guarantees that a faulty or manipulated key never propagates to subsequent key derivation or authentication computations.
2. **IND-CPAF Security (Bounded Leakage):** Because the KEM key pair in LOOM-AKE is strictly ephemeral, the adversary can observe at most one failure event per key pair through the handshake abort or retry behavior. This limits the observable failure leakage to a single bit for each ephemeral KEM instance. The IND-CPAF security notion explicitly captures this leakage and requires that the encapsulated secret remain indistinguishable from random even in the presence of the corresponding failure observation. Consequently, when the key derivation function is modeled as a random oracle or secure extractor, the derived session key remains pseudorandom for all successfully completed sessions.

Through the combination of rigidity and IND-CPAF security, we prove that LOOM-AKE remains secure in the Canetti–Krawczyk (CK) model [12, 13] despite a controlled, non-negligible DFR. This demonstrates that imperfect-correctness KEMs can be safely used in ephemeral authenticated key exchange to obtain significant bandwidth savings, provided that failures are explicitly detected and incorporated into the security model.

3.4 Architectural Paradigm: Sign-and-MAC vs. Multi-KEM

A fundamental design decision in post-quantum authenticated key exchange (AKE) is the choice of the cryptographic paradigm for peer authentication. Broadly, two paradigms exist for constructing lattice-based AKE: the *Multi-KEM* paradigm (typically employing three KEM instances, where static KEM keys act as long-term credentials) [11] and the *Sign-and-MAC* paradigm [37] (where ephemeral key exchange is decoupled from long-term signature authentication and symmetric key confirmation). We argue that the Sign-and-MAC paradigm, as realized in LOOM-AKE, possesses several systemic and architectural advantages over Multi-KEM alternatives in practical network deployments.

1. Infrastructure Compatibility and PKI Transition. The most significant barrier to deploying Multi-KEM protocols is their incompatibility with existing Public Key Infrastructure (PKI). In a Multi-KEM protocol (e.g., a static-ephemeral DH-like KEM hybrid), the long-term identity credentials of the communication peers must be static KEM public keys. Consequently, existing certificate authorities (CAs) and standard X.509 certificate formats—which are universally built around digital signatures (such as RSA, ECDSA, and now ML-DSA)—cannot be reused. Transitioning to a Multi-KEM architecture requires CAs to support static KEM certificates, a major disruption to legacy systems.

In contrast, LOOM-AKE’s Sign-and-MAC structure natively preserves the role of digital signatures (instantiated via Shuttle) for identity authentication. This allows the protocol to seamlessly integrate with existing PKI certificate chains. During the post-quantum transition, deployments can leverage hybrid certificates (combining classical and post-quantum signatures) without altering the core certificate validation logic of the network stack.

2. Robust Perfect Forward Secrecy (PFS) and Key Isolation. In the Multi-KEM paradigm, the long-term static KEM private key is directly involved in the mathematical derivation of the session key (e.g., via decapsulating a ciphertext sent by the peer). If an adversary records the handshake transcripts and subsequently compromises the static private key years later, any flaw in the protocol’s wPFS (weak Perfect Forward Secrecy) logic or key derivation could expose historical session keys.

LOOM-AKE enforces a strict separation of concerns through cryptographic key isolation. The ephemeral KEM (LOOM-KEM) is solely responsible for generating the raw shared secret K , and its ephemeral private key dk_{KEM} is immediately erased from memory upon decapsulation. The long-term Shuttle signature key is never used to encrypt, decrypt, or derive symmetric keying material; its sole purpose is to sign the role-separated handshake transcripts (DTBS_I and DTBS_R) to guarantee authenticity. Even if a long-term signature key is compromised in the future, the adversary gains zero mathematical leverage to decrypt past recorded traffic, guaranteeing robust, unconditional Perfect Forward Secrecy.

3. MTU-Friendly Handshake Scheduling and DoS Mitigation. Lattice-based post-quantum primitives suffer from a well-known inflation of public key and ciphertext sizes. In a 3-KEM protocol, the initiator must transmit multiple KEM elements (e.g., an ephemeral public key and a static ciphertext) in the very first flight. This heavily inflates the size of the unauthenticated initial packets, frequently exceeding the standard 1500-byte Ethernet MTU and triggering IP-layer fragmentation before any cryptographic association is established. This exposes the responder to severe reassembly-buffer Denial-of-Service (DoS) attacks.

LOOM-AKE resolves this scheduling conflict by delaying large payloads:

- **Phase 1 (Key Exchange):** The first two flights carry only a single ephemeral KEM payload (pk_{KEM} and ct_{KEM}) and nonces. As analyzed in Section 3.2, this keeps the initial flights comfortably under the 1500-byte MTU, ensuring fragmentation-free transmission during the unauthenticated phase.
- **Phase 2 (Authentication):** The larger Shuttle signatures, identities, and MAC tags are transmitted in flights 3 and 4. Crucially, these flights are transmitted *after* the ephemeral KEM exchange has completed. The payloads can thus be encrypted and fragmented at the application layer under keys derived from the ephemeral secret. The responder only allocates reassembly buffers for fragments that present a valid MAC tag, neutralizing unauthenticated DoS amplification.

4. Cryptographic Decoupling and Modular Security Proofs. In Multi-KEM constructions, the security reduction must simultaneously handle the IND-CCA security of multiple

static and ephemeral KEM instances, often leading to complex, tight algebraic bindings that make it difficult to replace individual components.

LOOM-AKE’s Sign-and-MAC architecture decouples the confidentiality of the key exchange from the unforgeability of the identity binding. This modularity simplifies the security reduction (as shown in Section 4) and enables algorithm-independent agility: the underlying ephemeral KEM or long-term signature scheme can be independently upgraded or replaced (e.g., substituting Shuttle with ML-DSA) without invalidating the high-level AKE security reduction, provided the individual components satisfy their respective standard security definitions (IND-CPAF for the KEM, EUF-CMA for the signature and MAC).

4 Security Statement and Analysis

4.1 Theoretical Security

In this subsection, we present a comprehensive, modular security analysis of the LOOM family. Our theoretical framework is structured hierarchically in a bottom-up manner:

1. **Hardness Foundations:** We first define our core algebraic assumptions, namely the standard decisional Module Learning with Rounding (MLWR) problem and our dedicated *Shifted-MLWR* problem (Subsubsection 4.1.1).
2. **Underlying PKE Security:** We prove that our encryption core, LOOM-CPAPKE, achieves IND-CPA security by reducing it to the hardness of MLWR and Shifted-MLWR (Subsubsection 4.1.2).
3. **Rigid IND-CPAF KEM Construction:** We analyze the random-oracle transformation that lifts our IND-CPA-secure PKE to a rigid, IND-CPAF-secure KEM (LOOM-CPAFKEM), which formally captures and bounds the side-channel leakage of decryption failures (Subsubsection 4.1.3).
4. **Protocol-Level SK-Security:** Finally, we define the post-specified-peer Canetti–Krawczyk (CK) model (Subsubsection 4.1.4) and prove that the complete LOOM-AKE protocol achieves session-key security (SK-security) and key consistency under active network adversaries, despite a non-negligible decryption failure rate (Subsubsection 4.1.5).

4.1.1 Hardness Assumptions

We first recall the standard decisional Module Learning with Rounding assumption. Throughout, $\text{MLWR}_{k,\ell,\eta,d}$ denotes the problem where the public matrix has ℓ rows and k columns (i.e., the secret vector has dimension k and the output has dimension ℓ).

Definition 4.1 (Decisional MLWR Assumption). *Let $k, \ell \in \mathbb{N}$, let η be the parameter of the centered binomial distribution, and let d be a compression precision. The decisional $\text{MLWR}_{k,\ell,\eta,d}$ problem is to distinguish between the following two distributions:*

- **Real:** sample $\mathbf{A} \leftarrow \$\mathcal{R}_q^{\ell \times k}$ and $\mathbf{s} \leftarrow \text{CBD}_\eta^k$, and output $(\mathbf{A}, \text{Compress}_q(\mathbf{A}\mathbf{s}, d))$.
- **Random:** sample $\mathbf{A} \leftarrow \$\mathcal{R}_q^{\ell \times k}$ and $\mathbf{u} \leftarrow \$\mathcal{R}_q^\ell$, and output $(\mathbf{A}, \text{Compress}_q(\mathbf{u}, d))$.

For any adversary $\mathcal{B}$, its distinguishing advantage is denoted $\text{Adv}_{k,\ell,\eta,d}^{\text{MLWR}}(\mathcal{B})$.

We next define the shifted variant needed for the second ciphertext component.

Definition 4.2 (Decisional Shifted-MLWR Assumption). *Let $k, \ell \in \mathbb{N}$ with $k \leq \ell$, let η be the parameter of the centered binomial distribution, let d_1 and d_2 be compression precisions, and let $\Delta \in \mathcal{R}_q^\ell$ be a public offset. For any vector $\mathbf{x} \in \mathcal{R}_q^\ell$, write $\mathbf{x} = (\mathbf{x}_{\leq k}, \mathbf{x}_{>k})$ with $\mathbf{x}_{\leq k} \in \mathcal{R}_q^k$ and $\mathbf{x}_{>k} \in \mathcal{R}_q^{\ell-k}$, and define*

$$\text{Compress}_{q,d_1,d_2}(\mathbf{x}) := (\text{Compress}_q(\mathbf{x}_{\leq k}, d_1), \text{Compress}_q(\mathbf{x}_{>k}, d_2)).$$

The decisional Shifted-MLWR $_{k,\ell,\eta,(d_1,d_2),\Delta}$ problem is to distinguish between the following two distributions:

- **Real:** sample $\mathbf{A} \leftarrow \$\mathcal{R}_q^{\ell \times k}$ and $\mathbf{s} \leftarrow \text{CBD}_\eta^k$, and output $(\mathbf{A}, \text{Compress}_{q,d_1,d_2}(\mathbf{A}\mathbf{s} + \Delta))$.
- **Random:** sample $\mathbf{A} \leftarrow \$\mathcal{R}_q^{\ell \times k}$ and $\mathbf{u} \leftarrow \$\mathcal{R}_q^\ell$, and output $(\mathbf{A}, \text{Compress}_{q,d_1,d_2}(\mathbf{u} + \Delta))$.

For any adversary $\mathcal{B}$, its distinguishing advantage is denoted $\text{Adv}_{k,\ell,\eta,(d_1,d_2),\Delta}^{\text{Shifted-MLWR}}(\mathcal{B})$.

Security Equivalence of Shifted-MLWR The decisional Shifted-MLWR $_{k,\ell,\eta,(d_1,d_2),\Delta}$ problem is computationally as hard as the standard decisional MLWR $_{k,\ell,\eta,(d_1,d_2)}$ problem. From an algebraic perspective, the introduction of a public, fixed offset $\Delta \in \mathcal{R}_q^\ell$ acts merely as a deterministic translation in the torus, which preserves both the geometric structure of the underlying lattice and the statistical distribution of the rounding noise. Specifically, since the secret vector $\mathbf{s}$ is sampled independently of Δ , the addition of Δ before rounding does not leak any additional entropy of the secret $\mathbf{s}$. Furthermore, as demonstrated by the formal reduction in [17] (following the Liu–Wang framework [39]), the statistical closeness between the rounded-with-offset distribution and the uniform distribution is established prior to the deterministic rounding step. Since the translation map $\mathbf{x} \mapsto \mathbf{x} + \Delta$ is a bijection over $\mathcal{R}_q^\ell$, it maps a uniformly random vector $\mathbf{u} \in \mathcal{R}_q^\ell$ to another perfectly uniform vector $\mathbf{u} + \Delta$. Consequently, any adversary capable of exploiting the public shift Δ to distinguish the real Shifted-MLWR sample from random can be immediately converted into a distinguisher for the standard MLWR (or MLWE) problem with the same advantage, confirming that the shifted variant introduces zero mathematical degradation to the cryptographic hardness.

4.1.2 Security Reduction of Loom-CPAPKE

A technical ingredient in the proof is the following joint-distribution property of randomized lifting.

Lemma 4.3 (Joint Distribution of Randomized Lifting). *Let $\mathbf{U} \leftarrow \$\mathcal{R}_q^k$, let $\mathbf{Y} := \text{Compress}_q(\mathbf{U}, d_t)$, and let $\mathbf{U}' \leftarrow \text{Inv}(\mathbf{Y}, d_t)$, where Inv samples uniformly from the coefficient-wise preimage bucket. Then the joint distribution of $(\mathbf{Y}, \mathbf{U}')$ is identical to the joint distribution of $(\text{Compress}_q(\mathbf{U}, d_t), \mathbf{U})$. Equivalently,*

$$(\mathbf{Y}, \mathbf{U}') \equiv (\text{Compress}_q(\mathbf{U}, d_t), \mathbf{U}).$$

Proof. It suffices to prove the claim coefficient-wise. Fix any $a \in \mathbb{Z}_q$ and $y \in \mathbb{Z}_{2d_t}$. If $\text{Compress}_q(a, d_t) \neq y$, then both joint probabilities are zero. Otherwise,

$$\Pr[Y = y, U' = a] = \Pr[Y = y] \cdot \Pr[U' = a \mid Y = y].$$

Since $Y = \text{Compress}_q(U, d_t)$ with $U \leftarrow \$\mathbb{Z}_q$, we have

$$\Pr[Y = y] = \frac{|\text{Preim}_q(y, d_t)|}{q},$$

and since $\text{Inv}(y, d_t)$ samples uniformly from $\text{Preim}_q(y, d_t)$,

$$\Pr[U' = a \mid Y = y] = \frac{1}{|\text{Preim}_q(y, d_t)|}.$$

Hence

$$\Pr[Y = y, U' = a] = \frac{1}{q}.$$

On the other hand,

$$\Pr[\text{Compress}_q(U, d_t) = y, U = a] = \Pr[U = a] = \frac{1}{q},$$

whenever $\text{Compress}_q(a, d_t) = y$. Therefore the joint distributions are identical. Applying this argument independently coefficient-wise proves the vector case. $\square$

We next prove the IND-CPA security of LOOM.CPAPKE via a reduction to the MLWR and Shifted-MLWR assumptions.

Theorem 4.4 (IND-CPA Security of LOOM PKE). *Let LOOM.CPAPKE be the public-key encryption scheme described in Algorithms 10 and 11. For any quantum polynomial-time adversary $\mathcal{A}$, there exist quantum polynomial-time adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that*

$$\text{Adv}_{\text{LOOM}}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}_{k, k, \eta_1, d_t}^{\text{MLWR}}(\mathcal{B}_1) + \text{Adv}_{k, k+1, \eta_2, (d_u, d_v), \Delta_b}^{\text{Shifted-MLWR}}(\mathcal{B}_2),$$

where

$$\Delta_b = \begin{pmatrix} \mathbf{0} \\ \mu_b \end{pmatrix} \in \mathcal{R}_q^{k+1}$$

Proof. We prove the theorem through a sequence of games. For each i , let S_i denote the event that $\mathcal{A}$ correctly guesses the hidden challenge bit in Game i .

Game 0 (Real Experiment). This is the real IND-CPA experiment. The challenger runs the genuine key-generation algorithm, generating $\mathbf{A}$ and $\mathbf{s} \leftarrow \text{CBD}_{\eta_1}^k$, and sets

$$pk = \text{Compress}_q(\mathbf{A}\mathbf{s}, d_t).$$

Upon receiving two challenge messages μ_0, μ_1 , it picks $b \leftarrow \$\{0, 1\}$ and encrypts μ_b honestly using fresh coins seed_1 . By definition,

$$\text{Adv}_{\text{LOOM}}^{\text{IND-CPA}}(\mathcal{A}) = |\Pr[S_0] - \frac{1}{2}|.$$

Game 1 (Replace Public Key with Uniform). We modify key generation. Instead of computing $\mathbf{t} = \mathbf{A}\mathbf{s}$, the challenger samples $\mathbf{b} \leftarrow \$\mathcal{R}_q^k$ uniformly at random and sets $pk := \text{Compress}_q(\mathbf{b}, d_t)$. All other steps remain unchanged.

The public-key distribution in Game 0 is $(\mathbf{A}, \text{Compress}_q(\mathbf{A}\mathbf{s}, d_t))$, whereas in Game 1 it is $(\mathbf{A}, \text{Compress}_q(\mathbf{b}, d_t))$. Any distinguisher between Game 0 and Game 1 therefore yields a distinguisher for $\text{MLWR}_{k, k, \eta_1, d_t}$. Hence there exists a quantum polynomial-time adversary $\mathcal{B}_1$ such that

$$|\Pr[S_1] - \Pr[S_0]| \leq \text{Adv}_{k, k, \eta_1, d_t}^{\text{MLWR}}(\mathcal{B}_1).$$

Game 2 (Replace Randomized Lift by a Fresh Uniform Lift). In Game 1, during encryption the challenger computes the lifted public key as

$$\mathbf{t}_{\text{lift}} \leftarrow \text{Inv}(pk, d_t; \text{PRF}(\text{seed}_1, \text{ctr}_0)),$$

and the ephemeral secret as

$$\mathbf{r}[i] \leftarrow \text{CBD}_{\eta_2}(\text{PRF}(\text{seed}_1, \text{ctr}_i)), \quad i = 1, \dots, k,$$

where the counter values $\text{ctr}_0 = 0$ and $\text{ctr}_1, \dots, \text{ctr}_k \geq 1$ are distinct. Since the PRF outputs on distinct inputs are computationally independent, the lifting randomness and the randomness used to sample $\mathbf{r}$ may be treated as independent.

Now in Game 1 we have $pk = \text{Compress}_q(\mathbf{b}, d_t)$ for a uniformly random $\mathbf{b} \in \mathcal{R}_q^k$. By Lemma 4.3, the joint distribution of

$$(pk, \mathbf{t}_{\text{lift}}) = (\text{Compress}_q(\mathbf{b}, d_t), \text{Inv}(\text{Compress}_q(\mathbf{b}, d_t), d_t))$$

is identical to the joint distribution of

$$(\text{Compress}_q(\mathbf{T}, d_t), \mathbf{T}), \quad \mathbf{T} \leftarrow \$\mathcal{R}_q^k.$$

Therefore, without changing the joint distribution seen by $\mathcal{A}$, we may equivalently sample a fresh uniform vector $\mathbf{T} \leftarrow \$\mathcal{R}_q^k$, set $pk := \text{Compress}_q(\mathbf{T}, d_t)$, and use $\mathbf{T}$ directly in place of $\mathbf{t}_{\text{lift}}$ during encryption. Hence

$$\Pr[S_2] = \Pr[S_1].$$

Game 3 (Replace the Challenge Ciphertext via Shifted-MLWR). In Game 2, the uncompressed ciphertext components are

$$\mathbf{u} = \mathbf{A}^T \mathbf{r}, \quad v_0 = \mathbf{T}^T \mathbf{r},$$

and the challenge ciphertext is

$$c_1 = \text{Compress}_q(\mathbf{u}, d_u), \quad c_2 = \text{Compress}_q(v_0 + \mu_b, d_v),$$

where $\mathbf{r} \leftarrow \text{CBD}_{\eta_2}^k$. Define the stacked matrix

$$\mathbf{A}^* = \begin{pmatrix} \mathbf{A}^T \\ \mathbf{T}^T \end{pmatrix} \in \mathcal{R}_q^{(k+1) \times k},$$

and the public offset

$$\Delta_b = \begin{pmatrix} \mathbf{0} \\ \mu_b \end{pmatrix} \in \mathcal{R}_q^{k+1}.$$

Since $\mathbf{A}$ is sampled uniformly at random, $\mathbf{A}^T$ is also uniform over $\mathcal{R}_q^{k \times k}$. Moreover, $\mathbf{T}$ is an independent uniform vector in $\mathcal{R}_q^k$, so $\mathbf{A}^*$ is uniformly distributed over $\mathcal{R}_q^{(k+1) \times k}$.

The pair (c_1, c_2) is precisely the compression of

$$\mathbf{A}^* \mathbf{r} + \Delta_b = \begin{pmatrix} \mathbf{A}^T \mathbf{r} \\ \mathbf{T}^T \mathbf{r} + \mu_b \end{pmatrix},$$

with compression precisions (d_u, d_v) , namely d_u on the first k coordinates and d_v on the last coordinate, respectively. Therefore the challenge ciphertext distribution in Game 2 is an instance of $\text{Shifted-MLWR}_{k, k+1, \eta_2, (d_u, d_v), \Delta_b}$.

In Game 3, the challenger replaces the challenge ciphertext with

$$c_1 := \text{Compress}_q(\mathbf{u}^*, d_u), \quad c_2 := \text{Compress}_q(v^* + \mu_b, d_v),$$

where $\mathbf{u}^* \leftarrow \$\mathcal{R}_q^k$ and $v^* \leftarrow \$\mathcal{R}_q$ are uniformly random. Hence there exists a quantum polynomial-time adversary $\mathcal{B}_2$ such that

$$|\Pr[S_3] - \Pr[S_2]| \leq \text{Adv}_{k, k+1, \eta_2, (d_u, d_v), \Delta_b}^{\text{Shifted-MLWR}}(\mathcal{B}_2).$$

Game 4 (Challenge Ciphertext is Independent of b). This game is identical to Game 3; we introduce it only to record the following observation. Since v^* is uniformly distributed over $\mathcal{R}_q$, for every fixed message offset $\mu_b \in \mathcal{R}_q$, the shifted value $v^* + \mu_b$ is also uniformly distributed over $\mathcal{R}_q$. Therefore the distribution of $c_2 = \text{Compress}_q(v^* + \mu_b, d_v)$ is independent of the hidden bit b . Likewise, $c_1 = \text{Compress}_q(u^*, d_u)$ is already independent of b . Hence the entire challenge ciphertext carries no information about b , and

$$\Pr[S_4] = \frac{1}{2}.$$

Since Game 4 is identical to Game 3, we also have $\Pr[S_4] = \Pr[S_3]$.

Conclusion. By the triangle inequality applied to the full game sequence $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$,

$$\begin{aligned} \left| \Pr[S_0] - \frac{1}{2} \right| &= |\Pr[S_0] - \Pr[S_4]| \\ &\leq |\Pr[S_0] - \Pr[S_1]| + |\Pr[S_1] - \Pr[S_2]| + |\Pr[S_2] - \Pr[S_3]| + |\Pr[S_3] - \Pr[S_4]|. \end{aligned}$$

Substituting $\Pr[S_2] = \Pr[S_1]$ (Game 2), $\Pr[S_4] = \Pr[S_3]$ (Game 4 is identical to Game 3), and the bounds from Game 1 and Game 3 yields

$$\text{Adv}_{\text{LOOM}}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}_{k,k,\eta_1,d_t}^{\text{MLWR}}(\mathcal{B}_1) + \text{Adv}_{k,k+1,\eta_2,(d_u,d_v),\Delta_b}^{\text{Shifted-MLWR}}(\mathcal{B}_2).$$

This completes the proof. $\square$

4.1.3 IND-CPAF and Rigidity KEM

Standard MLWE-based KEMs are typically designed to achieve IND-CCA security by enforcing a negligible Decryption Failure Rate (DFR). However, to achieve extreme compactness, our underlying KEM deliberately allows a non-negligible DFR (e.g., $\delta \approx 2^{-20}$). In a real-world active AKE deployment, this controlled correctness error introduces a severe security vulnerability if analyzed only under passive models. Specifically, an active adversary can inject manipulated ciphertexts into the handshake; because the protocol's subsequent execution (i.e., whether it proceeds or aborts) is observable, the adversary effectively gains access to a *Failure Oracle*. This oracle leaks a binary success/failure relation that is geometrically correlated with the boundary of the decapsulation secret's error distribution. Standard IND-CPA security is fundamentally insufficient in this scenario, as its security game assumes a purely passive adversary and does not guarantee key pseudorandomness (or secret confidentiality) conditioned on such active failure queries. Armed with this 1-bit feedback per session, an adversary could perform failure-boosting or geometric pruning attacks to reconstruct the secret key. Consequently, we must transition to a stronger security notion, **IND-CPAF**, which explicitly incorporates this bounded failure-leakage channel into the security game.

To formalize the security requirements for our KEM, we define two properties: **IND-CPAF** and **Rigidity**.

Definition 4.5 (IND-CPAF Security). *The IND-CPAF game models an adversary $\mathcal{A}$ that attempts to distinguish the true encapsulated key from a random key, given access to a restricted Failure Oracle $\mathcal{O}_{\text{fail}}$. The game proceeds as follows:*

1. **Setup:** *The challenger generates $(pk, sk) \leftarrow \text{KGen}(1^\lambda)$ and gives pk to $\mathcal{A}$.*
2. **Challenge:** *The challenger computes $(K_e, ct^*) \leftarrow \text{Encap}(pk)$ and $K_d \leftarrow \text{Decap}(sk, ct^*)$. It samples a random bit $b \leftarrow \{0, 1\}$. If $b = 0$, it sets $K_b = K_e$; otherwise, it samples a random key K_1 and sets $K_b = K_1$. The challenger gives (ct^*, K_b) to $\mathcal{A}$.*
3. **Query:** *$\mathcal{A}$ can query the Failure Oracle $\mathcal{O}_{\text{fail}}(ct)$. If $ct \neq ct^*$, the oracle returns $\perp$. If $ct = ct^*$, the oracle returns 1 if $K_d = K_e$ (success), and 0 if $K_d \neq K_e$ (failure).*

4. **Guess:** $\mathcal{A}$ outputs a guess b' .

The advantage of $\mathcal{A}$ is defined as $\text{Adv}_{\text{KEM}}^{\text{IND-CPAF}}(\mathcal{A}) = |\Pr[b' = b] - 1/2|$. A KEM is IND-CPAF secure if this advantage is negligible for all probabilistic polynomial-time adversaries.

Definition 4.6 (Rigidity, KEM). A KEM scheme $(\text{KGen}, \text{Encap}, \text{Decap})$ is ε -rigid if

$$\mathbb{E}_{(\text{pk}, \text{sk}) \leftarrow \text{KGen}} \left[\Pr \left[K_d \neq \perp \wedge K_d \neq K_e \mid \begin{array}{l} (K_e, ct) \leftarrow \text{Encap}(\text{pk}) \\ K_d = \text{Decap}(\text{sk}, ct) \end{array} \right] \right] \leq \varepsilon.$$

Under rigidity, any decryption error results in $\perp$, protecting the protocol layer from ever using an unpredictable, potentially insecure key. Together with IND-CPAF, this ensures that every completed session derives its keys from a pseudorandom and correct shared secret.

We now show that the KEM used in our AKE construction, denoted LOOM.KEM , satisfies the two properties required by the higher-level protocol proof, namely *Rigidity* and IND-CPAF security. The proof is modular. We first prove that, assuming the underlying PKE scheme is IND-CPA secure, the tagged KEM construction is both rigid and IND-CPAF secure in the random-oracle model. We then instantiate this assumption with Theorem 4.4, which reduces the IND-CPA security of LOOM.CPAPKE to the decisional MLWR and Shifted-MLWR assumptions.

The proof proceeds through an intermediate tagged PKE scheme. Let $\mathcal{E}$ denote the underlying PKE scheme, instantiated by LOOM.CPAPKE ; let $\mathcal{E}'$ denote the tagged PKE obtained by appending a tag generated from F ; and let $\mathcal{E}^*$ denote the resulting KEM, instantiated by LOOM.CPAFKEM . The PRF used for tagging is denoted by F , and the key-derivation random oracle is denoted by H .

At a high level, the argument consists of three steps. First, the tag check makes the intermediate PKE $\mathcal{E}'$ rigid except with the birthday collision probability of F . Second, the one-wayness with failure of $\mathcal{E}'$ follows from the IND-CPA security of the underlying PKE $\mathcal{E}$ in the random-oracle model. Third, the OW-CPAF security of $\mathcal{E}'$ implies the IND-CPAF security of the KEM $\mathcal{E}^*$, because any adversary distinguishing the encapsulated key from random must query the key-derivation oracle H on the encapsulated message.

Lemma 4.7 ($\text{PKE} \xrightarrow{\text{ROM}} \text{KEM rigid}$). For any rigidity adversary $\mathcal{D}$ against the PKE scheme $\mathcal{E}'$ that issues at most q_{F} queries to the PRF F , whose range space has size $|\mathcal{T}|$, the rigidity advantage is bounded by

$$\varepsilon := \text{Adv}_{\mathcal{E}', \mathcal{D}}^{\text{rigid}}(\lambda) \leq \frac{q_{\text{F}}(q_{\text{F}} - 1)}{2|\mathcal{T}|}.$$

Moreover, the rigidity of the resulting KEM scheme $\mathcal{E}^*$ is inherited from the rigidity of $\mathcal{E}'$.

Proof. In the tagged PKE scheme $\mathcal{E}'$, any decrypted value $m' = \text{Dec}(\text{sk}, c = \text{Enc}(\text{pk}, m))$ must satisfy

$$\text{F}_{m'}(c) = \text{F}_m(c)$$

in order to be accepted as a non-abort value. Therefore, for an incorrect but non-aborting plaintext $m' \neq m$ to be output, the adversary must find a collision in the output of the PRF F on two distinct inputs involving the same ciphertext. Since the adversary makes at most q_{F} queries to F , the birthday bound gives

$$\text{Adv}_{\mathcal{E}', \mathcal{D}}^{\text{rigid}}(\lambda) \leq \frac{q_{\text{F}}(q_{\text{F}} - 1)}{2|\mathcal{T}|}.$$

This rigidity property carries over directly to the KEM $\mathcal{E}^*$. Indeed, as long as the event $m' \neq \perp$ and $m' \neq m$ is excluded, every non-aborting decapsulation output is derived from the same message as in encapsulation. Hence, if

$$K_e = \text{H}(m) \quad \text{and} \quad K_d = \text{H}(m'),$$

then whenever $m' \neq \perp$, rigidity implies $m' = m$, and therefore $K_d = K_e$. Thus, except with probability ε , decapsulation either returns the intended key or aborts. $\square$

We next relate the one-wayness-with-failure security of the tagged PKE $\mathcal{E}'$ to the IND-CPA security of the underlying PKE $\mathcal{E}$.

Lemma 4.8 (PKE IND-CPA $\xRightarrow{\text{ROM}}$ PKE OW-CPAF). *If the PKE scheme $\mathcal{E}$ is δ -correct, then so is $\mathcal{E}'$. For any OW-CPAF adversary $\mathcal{A}$ that issues at most q_F queries to the PRF F , there exists an IND-CPA adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\mathcal{E}', \mathcal{A}}^{\text{ow-cpaf}}(\lambda) \leq \left(1 - \frac{q_F}{|\mathcal{T}|}\right)^{-1} \cdot 4\text{Adv}_{\mathcal{E}, \mathcal{B}}^{\text{IND-CPA}}(\lambda).$$

Proof. Correctness of $\mathcal{E}'$ follows from the fact that the transformation does not change the behavior of correct decryption. If the original PKE decryption returns the correct message m , the tag check also accepts. If decryption returns an incorrect value $m' \neq m$, the additional tag check accepts only when

$$F_{m'}(c) = F_m(c) = t,$$

which happens only with the random-oracle collision probability bounded above. Thus, the tag transformation does not introduce additional correctness errors except for the negligible collision event already accounted for by rigidity.

We now prove the security reduction. Let G_0 be the real OW-CPAF game for the adversary $\mathcal{A}$ against $\mathcal{E}'$. In the first hop, define Game G_1 as follows. The only difference from G_0 is that the dependence of the failure-check oracle $\mathcal{O}_{fail}$ on the secret key sk is removed. Instead, the simulator tosses a random coin b^* to guess the correct output of $\mathcal{O}_{fail}(\text{ct}^*)$. When this guess is correct, G_1 is identical to G_0 from the adversary's perspective. Hence,

$$\Pr[G_0^{\mathcal{A}} = 1] \leq 2 \Pr[G_1^{\mathcal{A}} = 1].$$

We then construct an IND-CPA adversary $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$ against $\mathcal{E}$. The adversary $\mathcal{B}_1$ samples two independent challenge messages

$$m_0, m_1 \leftarrow \$\mathbb{M}$$

and submits them to the IND-CPA challenger. It receives the challenge ciphertext

$$c_b^* = \mathcal{E}.\text{Enc}(\text{pk}, m_b).$$

Using m_0 and m_1 together with c_b^* , it constructs two tagged ciphertexts for $\mathcal{E}'$:

$$\bar{c}_0 = c_b^* \parallel F_{m_0}(c_b^*), \quad \bar{c}_1 = c_b^* \parallel F_{m_1}(c_b^*).$$

The adversary $\mathcal{B}$ then invokes $\mathcal{A}$ twice, once on $(\text{pk}, \bar{c}_0)$ and once on $(\text{pk}, \bar{c}_1)$, while simulating the oracles F and $\mathcal{O}_{fail}$.

If $b = 0$, then the first invocation corresponds to the real G_1 challenge, whereas the second invocation requires $\mathcal{A}$ to recover m_1 from a ciphertext whose underlying PKE component encrypts m_0 . Since m_0 and m_1 are independently sampled, this can only succeed by guessing a preimage of the random-oracle tag, and hence

$$\Pr[\mathcal{A}^{F, \mathcal{O}_{fail}}(\text{pk}, \bar{c}_1) = m_1] \leq \frac{q_F}{|\mathcal{T}|}.$$

The case $b = 1$ is symmetric: the second invocation corresponds to the real G_1 challenge, while the first invocation is bounded by the same preimage probability.

The adversary $\mathcal{B}$ outputs its guess as follows. If only the first invocation returns m_0 , then $\mathcal{B}$ outputs $b' = 0$. If only the second invocation returns m_1 , then $\mathcal{B}$ outputs $b' = 1$. Otherwise, it outputs a uniform random bit.

Therefore, regardless of the challenge bit b , the adversary $\mathcal{A}$ is effectively challenged once in the real G_1 experiment and once in an independent random-oracle preimage experiment. Consequently,

$$\begin{aligned}\text{Adv}_{\mathcal{E},\mathcal{B}}^{\text{IND-CPA}}(\lambda) &= \left| \Pr[b' = b] - \frac{1}{2} \right| \\ &\geq \frac{1}{2} \cdot \Pr[G_1 = 1] \cdot \left(1 - \frac{q_F}{|\mathcal{T}|} \right).\end{aligned}$$

Rearranging gives

$$\Pr[G_1 = 1] \leq 2\text{Adv}_{\mathcal{E},\mathcal{B}}^{\text{IND-CPA}}(\lambda) \cdot \left(1 - \frac{q_F}{|\mathcal{T}|} \right)^{-1}.$$

Combining this with $\Pr[G_0^{\mathcal{A}} = 1] \leq 2\Pr[G_1^{\mathcal{A}} = 1]$, we obtain

$$\text{Adv}_{\mathcal{E}',\mathcal{A}}^{\text{ow-cpaf}}(\lambda) = \Pr[G_0^{\mathcal{A}} = 1] \leq \left(1 - \frac{q_F}{|\mathcal{T}|} \right)^{-1} \cdot 4\text{Adv}_{\mathcal{E},\mathcal{B}}^{\text{IND-CPA}}(\lambda).$$

This completes the proof. $\square$

It remains to lift the OW-CPAF security of the tagged PKE to the IND-CPAF security of the KEM. This step is standard in the random-oracle model: if an adversary distinguishes the derived key $H(m^*)$ from random, then it must have queried H on the underlying message m^* .

Lemma 4.9 (PKE OW-CPAF $\xRightarrow{\text{ROM}}$ KEM IND-CPAF). *If PKE $\mathcal{E}'$ is δ -correct, then KEM $\mathcal{E}^*$ is $(\delta + \varepsilon)$ -correct, where ε is the rigidity bound from Lemma 4.7. For any IND-CPAF adversary $\mathcal{A}$ against $\mathcal{E}^*$ issuing at most q_H queries to the random oracle H , there exists an OW-CPAF adversary $\tilde{\mathcal{A}}$ against $\mathcal{E}'$ such that*

$$\text{Adv}_{\mathcal{E}^*,\mathcal{A}}^{\text{ind-cpaf}}(\lambda) \leq q_H \cdot \text{Adv}_{\mathcal{E}',\tilde{\mathcal{A}}}^{\text{ow-cpaf}}(\lambda).$$

Moreover, the running time of $\tilde{\mathcal{A}}$ is approximately that of $\mathcal{A}$.

Proof. Suppose an adversary $\mathcal{A}$ wins the IND-CPAF game against $\mathcal{E}^*$ with non-negligible advantage. We construct an adversary $\tilde{\mathcal{A}}$ against the OW-CPAF security of $\mathcal{E}'$ by simulating the IND-CPAF game for $\mathcal{A}$.

Let m^* be the encapsulated message underlying the challenge ciphertext ct^* . Define the event **Flag** to be the event that $\mathcal{A}$ queries the random oracle H on input m^* . If **Flag** does not occur, then the real key

$$K_0 = H(m^*)$$

is information-theoretically hidden from $\mathcal{A}$, and is indistinguishable from an independently sampled random key

$$K_1 \leftarrow \mathcal{K}.$$

Thus, conditioned on $\neg\text{Flag}$, the adversary can guess the challenge bit only with probability $1/2$:

$$\Pr\left[\mathcal{A}^{H,\mathcal{O}_{\text{fail}}}(\text{pk}, \text{ct}^*, K_b) = b \mid \neg\text{Flag}\right] = \frac{1}{2}.$$

It follows that

$$\text{Adv}_{\mathcal{E}^*,\mathcal{A}}^{\text{ind-cpaf}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right| \leq \Pr[\text{Flag}].$$

Whenever **Flag** occurs, the message m^* appears in the query list $\mathcal{L}_H$ of the random oracle H . The adversary $\tilde{\mathcal{A}}$ therefore samples one element uniformly from $\mathcal{L}_H$ and outputs it as its guess for m^* . Since $\mathcal{A}$ makes at most q_H queries, this succeeds with probability at least

$$\frac{\Pr[\text{Flag}]}{q_H}.$$

Hence,

$$\frac{\Pr[\text{Flag}]}{q_H} \leq \text{Adv}_{\mathcal{E}', \tilde{\mathcal{A}}}^{\text{ow-cpaf}}(\lambda),$$

and therefore

$$\text{Adv}_{\mathcal{E}^*, \mathcal{A}}^{\text{ind-cpaf}}(\lambda) \leq \Pr[\text{Flag}] \leq q_H \cdot \text{Adv}_{\mathcal{E}', \tilde{\mathcal{A}}}^{\text{ow-cpaf}}(\lambda).$$

This proves the claim. $\square$

Theorem 4.10 (IND-CPAF Security and Rigidity of LOOM.CPAFKEM). *Assume that the tagging function F behaves as a secure PRF with range space $\mathcal{T}$, and that the key-derivation function H is modeled as a random oracle. Let $\mathcal{E}^* = \text{LOOM.CPAFKEM}$ be the KEM obtained from $\mathcal{E} = \text{LOOM.CPAPKE}$ via the tagged PKE transformation and the KEM construction described above.*

If LOOM.CPAPKE is δ -correct, then LOOM.CPAFKEM is $(\delta + \varepsilon)$ -correct and ε -rigid, where

$$\varepsilon \leq \frac{q_F(q_F - 1)}{2|\mathcal{T}|}.$$

Moreover, for any IND-CPAF adversary $\mathcal{A}$ against LOOM.CPAFKEM that makes at most q_H queries to H and at most q_F queries to F , there exist quantum polynomial-time adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that

$$\text{Adv}_{\text{LOOM.CPAFKEM}, \mathcal{A}}^{\text{ind-cpaf}}(\lambda) \leq 4q_H \cdot \left(1 - \frac{q_F}{|\mathcal{T}|}\right)^{-1} \cdot \left(\text{Adv}_{k, k, \eta_1, d_t}^{\text{MLWR}}(\mathcal{B}_1) + \text{Adv}_{k, k+1, \eta_2, (d_u, d_v), \Delta_b}^{\text{Shifted-MLWR}}(\mathcal{B}_2)\right),$$

where

$$\Delta_b = \begin{pmatrix} \mathbf{0} \\ \mu_b \end{pmatrix} \in \mathcal{R}_q^{k+1}.$$

Proof. The rigidity and correctness claims follow directly from Lemma 4.7. The IND-CPAF security bound is obtained by combining Lemma 4.9, which reduces the IND-CPAF security of $\mathcal{E}^*$ to the OW-CPAF security of $\mathcal{E}'$, with Lemma 4.8, which reduces the OW-CPAF security of $\mathcal{E}'$ to the IND-CPA security of $\mathcal{E}$. Instantiating $\mathcal{E}$ with LOOM.CPAPKE and applying Theorem 4.4 gives the stated reduction to the MLWR and Shifted-MLWR assumptions. $\square$

4.1.4 The Security Model: Canetti-Krawczyk (CK)

We analyze the LOOM-AKE protocol in the CK model for Authenticated Key Exchange (AKE). In this model, parties execute multiple instances of the protocol, termed *sessions*. The adversary (denoted as the SK-Attacker) has full control over the communication network (intercepting, delaying, dropping, or injecting messages) and can perform the following queries:

- **Session-State Reveal:** Learn the internal state of an incomplete session.
- **Session-Key Query:** Learn the session key of a completed session.
- **Party Corruption:** Take full control of a party, learning all its long-term secrets.

A session is activated with a local identity P , a session identifier s , and a destination d . Upon completion, it outputs (P, s, Q) and a session key K_{session} , where Q is the peer's identity. Two sessions are considered *matching* if they share the same session identifier and their local/peer identities are mirrored.

Formally, we adopt the *post-specified peer* (PSP) model as defined by Canetti and Krawczyk [13] to formalize the security of the KEM-based IKEv2 instantiation. This model is particularly suited for IKEv2-like protocols where the peer's identity is disclosed only after the initial exchange.

Sessions and Identifiers. A party $P \in \mathcal{U}$ can initiate multiple protocol instances, each referred to as a *session*. To avoid ambiguity between local management and protocol-level identity binding, we distinguish between the following:

- **(Local) Session ID:** A unique ‘ s ’ generated locally by party P to identify a session within its own internal state (e.g., the SPIs in IKEv2).
- **Session Triple (P, s, Q) :** The logical output of a completed session. It binds the local identity P and the local s to a specific peer Q . In the PSP setting, Q is initially set to a null value ($*$) and is only “post-specified” as a specific identity $Q \in \mathcal{U}$ upon the session’s completion.

Following Section 2.1 of [13], in the PSP setting, a key exchange session is activated at a party P with parameters (P, s) , where P is the local identity, s is a unique session identifier. When a session completes, it outputs a session key κ and a completion triple (P, s, Q) , where Q is the identity of the party with whom P believes it has exchanged the key.

Session Stage and Internal State. Each session belongs to a **stage** $\in \{\textit{incomplete}, \textit{completed}, \textit{expired}\}$ and maintains its own internal states:

- **Incomplete:** The session is ongoing. P holds an ephemeral internal state state_s .
- **Completed (Accepted):** The session derives a session key κ and outputs the triple (P, s, Q) . The internal state state_s is erased.
- **Expired:** The session key κ is erased, though the triple (P, s, Q) remains for definition purposes.

In the context of IKEv2, the internal state involves the algorithm randomness and secret exponent of the Diffie-Hellman exchange; and under the usage of post-quantum KEM, it also captures the KEM randomness and the ephemeral decapsulation key sk .

Matching Sessions. The notion of matching sessions is central to defining freshness and authentication. According to Definition 2 in [?], two sessions (P, s) and (Q, s) are called **matching** if they share the same unique session identifier s , and one of the following conditions holds:

1. Session (Q, s) is *incomplete* (i.e., has not yet output a completion triple); OR
2. Session (Q, s) has *completed* and output the triple (Q, s, P) .

This definition ensures that even if a peer has not yet finished its protocol execution, it can still be recognized as the matching counterpart to a completed session.

Adversarial Queries. The adversary $\mathcal{A}$ is a PPT algorithm that controls the communication network. Following the PSP framework, $\mathcal{A}$ interacts with parties through the following queries:

- **Send** $((P, s), m)$: Delivers message m to session s at party P .
- **Reveal-session-key** (P, s) : Returns κ for a *completed* session.
- **Reveal-state** (P, s) : Returns the internal ephemeral state of an *incomplete* session.
- **Corrupt** (P) : Returns the long-term secret signing key of party P .
- **Test** (P, s) : This query can only be issued to a *completed, fresh* session. If so, the challenger returns either κ or a random string based on a hidden bit $b \leftarrow_{\$} \{0, 1\}$.

SK-Security and Freshness. We restate the definition of SK-security and freshness as follows:

Definition 4.11 (SK-security [?]). *An attacker with the above capabilities is called an SK-attacker. An AKE protocol π is called SK-secure if for all SK-attackers $\mathcal{A}$ running against π it holds:*

- P1) Any two completed matching sessions in a run of protocol π under attacker $\mathcal{A}$, given that neither of their owners has been corrupted, will always derive the same session key output, except for a negligible probability.*
- P2) $\mathcal{A}$ succeeds (in its test-session distinguishing attack) with probability not more than $1/2$ plus a negligible fraction.*

Definition 4.12 (Freshness). *A session (P, s) that completed with triple (P, s, Q) is fresh if:*

- 1. From the view of party P :*
 - incomplete stage: no Reveal-state has been issued to (P, s) ;*
 - completed stage: no Reveal-session-key has been issued to (P, s, Q) .*
- 2. If a matching session for (P, s, Q) exists, which is (Q, s) or (Q, s, P) , it must also satisfy from the view of party Q :*
 - incomplete stage: no Reveal-state has been issued to (Q, s) ;*
 - completed stage: no Reveal-session-key has been issued to (Q, s, P) ;*
 - Under either incomplete or completed stage, no Corrupt has been issued to party Q .*

Forward Secrecy. In this model, *forward secrecy* is captured by the resilience of session keys to the corruption of long-term keys after the session has expired. Specifically, consider a session (P, s, Q) that has entered the *expired* status (at which point the session key κ is erased from P 's memory) and its matching session (Q, s) at a party Q . The model requires that even if an adversary $\mathcal{A}$ issues a **Corrupt**(P) query after (P, s, Q) has expired, the session key κ of the matching session (Q, s) – provided it remains *fresh* – remains indistinguishable from a random string. This ensures that the compromise of P 's long-term signing key does not affect the security of previously established and now expired session keys.

4.1.5 Security Reduction of Loom-AKE

We now prove that LOOM-AKE is SK-secure in the post-specified-peer Canetti–Krawczyk (CK) model. The proof follows the structure of the signature-based IKE/SIGMA analysis of [13], with the Diffie–Hellman challenge replaced by an ephemeral KEM challenge. The main additional ingredients are the ϵ -rigidity and IND-CPAF security of the underlying KEM, which account for explicit decapsulation failures.

Throughout this section, we write

$$\text{sid} := s_I \parallel s_R, \quad \text{salt} := N_I \parallel N_R.$$

The actual key derivation algorithm derives

$$(SS, mk_I, mk_R) \leftarrow \text{KDF}^+(K, \text{salt})$$

by the following chained and domain-separated KDF evaluations:

$$X_{SS} := \text{salt} \parallel 0\text{x}01,$$

$$X_I(SS) := SS \parallel \text{salt} \parallel 0\mathbf{x}02, \quad X_R(mk_I) := mk_I \parallel \text{salt} \parallel 0\mathbf{x}03,$$

and

$$SS = \text{KDF}(K, X_{SS}),$$

$$mk_I = \text{KDF}(K, X_I(SS)), \quad mk_R = \text{KDF}(K, X_R(mk_I)).$$

All KDF inputs are canonically encoded and domain separated. In the hybrid proof below, when only one directional MAC key is relevant, we write mk for that key and suppress the other direction from the notation. More precisely, mk denotes the directional MAC key used in the guessed authentication flow, and it is always derived according to the chained key schedule above.

Transcript authentication. Since the KDF input contains only the nonce-derived salt, transcript binding is provided by the signature layer. In the actual protocol, the data-to-be-signed strings are the following role-separated transcript strings:

$$\text{DTBS}_I = \text{'0'} \parallel \text{sid} \parallel ek \parallel ct_{\text{kem}},$$

and

$$\text{DTBS}_R = \text{'1'} \parallel \text{sid} \parallel ek \parallel ct_{\text{kem}}.$$

The string '0' denotes the initiator-authentication direction, and '1' denotes the responder-authentication direction. The signed strings bind the role of the signer, the session identifier, the ephemeral KEM public key, and the KEM ciphertext. The AKE proof requires ordinary existential unforgeability under chosen-message attacks for signatures on these role-separated data-to-be-signed strings. If the concrete signature instantiation is analyzed under the stronger sUF-CMA notion, then it satisfies the EUF-CMA assumption used here.

Single-use decapsulation requirement. The IND-CPAF security notion used in the KEM analysis models the leakage of at most one public accept/reject bit per ephemeral KEM key pair. Accordingly, LOOM-AKE enforces the following state-machine condition: each ephemeral decapsulation key dk generated by an initiator session is used to process at most one KEM ciphertext. After this decapsulation attempt, regardless of whether it succeeds or returns $\perp$, the ephemeral decapsulation state is erased and no further ciphertext is processed under the same dk . This condition is essential: the KEM is intended only for strictly ephemeral use and is not claimed to be secure under static or reused decapsulation keys.

PRF security of the KDF. In the proof, the key-derivation function is used as a keyed function family: for a key K and an input string X , the value $\text{KDF}(K, X)$ is the output of the KDF on input X under key K . We assume that the family

$$\{X \mapsto \text{KDF}(K, X)\}_K$$

is pseudorandom. That is, no efficient adversary can distinguish oracle access to $X \mapsto \text{KDF}(K, X)$, where $K \leftarrow \$ \mathcal{K}_{\text{kdf}}$, from oracle access to a uniformly random function $F : \mathcal{X} \rightarrow \mathcal{Y}$. The corresponding advantage is defined as

$$\text{Adv}_{\text{KDF}}^{\text{PRF}}(\mathcal{A}) := \left| \Pr \left[\mathcal{A}^{\text{KDF}(K, \cdot)} = 1 \right] - \Pr \left[\mathcal{A}^{F(\cdot)} = 1 \right] \right|.$$

MAC unforgeability. Let $\text{MAC} = (\text{Mac}, \text{Vrfy})$ be a message authentication code with key space $\mathcal{K}_{\text{mac}}$. We say that MAC is existentially unforgeable under chosen-message attacks, or EUF-CMA secure, if no efficient adversary with oracle access to $\text{Mac}_K(\cdot)$, where $K \leftarrow \$ \mathcal{K}_{\text{mac}}$, can output a valid pair (m^*, τ^*) such that m^* was never queried to the MAC oracle and $\text{Vrfy}_K(m^*, \tau^*) = 1$. The corresponding advantage is defined as

$$\text{Adv}_{\text{MAC}}^{\text{MAC}}(\mathcal{A}) := \Pr \left[\text{Vrfy}_K(m^*, \tau^*) = 1 \wedge m^* \notin Q_{\text{Mac}} : (m^*, \tau^*) \leftarrow \mathcal{A}^{\text{Mac}_K(\cdot)} \right],$$

where Q_{Mac} denotes the set of messages queried by $\mathcal{A}$ to the MAC oracle.

In the actual protocol, the MAC inputs are role-separated identity-binding strings under directional, session-specific MAC keys. Concretely, the authentication tags are computed as

$$\tau_I = \text{MAC}(mk_I, \text{'0'} \parallel \text{sid} \parallel ID_I),$$

and

$$\tau_R = \text{MAC}(mk_R, \text{'1'} \parallel \text{sid} \parallel ID_R).$$

The strings '0' and '1' separate the two authentication directions, while mk_I and mk_R are also directional MAC keys derived by KDF^+ using distinct domain-separated inputs. The signatures bind the key-exchange transcript $(\text{sid}, ek, ct_{\text{kem}})$, and the MAC tags bind the post-specified identities to the KEM-derived directional authentication keys. Thus the proof requires that the MAC keys are session-specific, directional, and pseudorandom, and that the MAC is EUF-CMA secure on the above role-separated identity strings.

Theorem 4.13 (SK-security of LOOM-AKE). *Assume that the underlying KEM is ϵ -rigid and IND-CPAF secure for single-use ephemeral decapsulation keys, that SIG is existentially unforgeable under chosen-message attacks (EUF-CMA), that MAC is EUF-CMA secure, and that the keyed function family induced by KDF is PRF-secure. Assume further that the chained KDF^+ schedule used to derive SS, mk_I, mk_R from the KEM shared secret and the nonce-derived salt uses domain-separated, injectively encoded inputs. Then LOOM-AKE is SK-secure in the post-specified-peer CK model.*

More precisely, for any SK-adversary $\mathcal{A}$ activating at most N_s sessions among N_p parties, there exist efficient adversaries $\mathcal{B}_1, \dots, \mathcal{B}_6$ such that

$$\begin{aligned} \text{Adv}_{\text{LOOM-AKE}}^{\text{SK}}(\mathcal{A}) &\leq N_s N_p \cdot \left(\text{Adv}_{\text{KEM}}^{\text{IND-CPAF}}(\mathcal{B}_1) + \text{Adv}_{\text{KDF}}^{\text{PRF}}(\mathcal{B}_2) \right. \\ &\quad \left. + \text{Adv}_{\text{KDF}}^{\text{PRF}}(\mathcal{B}_3) + \text{Adv}_{\text{KEM}}^{\text{IND-CPAF}}(\mathcal{B}_4) \right) \\ &\quad + \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{B}_5) + \text{Adv}_{\text{MAC}}^{\text{MAC}}(\mathcal{B}_6) + \epsilon + \text{negl}(\lambda). \end{aligned}$$

Proof structure. We prove Theorem 4.13 by establishing the two requirements of SK-security in Definition 4.11. First, we prove *P1*, namely key consistency for completed matching sessions owned by uncorrupted parties. Then, we prove *P2*, namely session-key secrecy for any fresh tested session, by a CK guessing argument and a sequence of hybrid games.

P1: key consistency. Consider two completed matching sessions owned by uncorrupted parties, with initiator I and responder R . By the definition of matching sessions, the two sessions agree on the session identifier

$$\text{sid} = s_I \parallel s_R$$

and on the first two key-exchange messages. In particular, they agree on the nonces N_I, N_R , the ephemeral KEM public key ek , and the KEM ciphertext ct_{kem} .

The responder computes

$$(K_e, ct_{\text{kem}}) \leftarrow \text{KEM.Encap}(ek),$$

whereas the initiator computes

$$K_d \leftarrow \text{KEM.Decap}(dk, ct_{\text{kem}}).$$

Unlike Diffie–Hellman, transcript agreement alone does not imply key agreement for a KEM with non-zero decapsulation failure probability. We therefore use ϵ -rigidity of the KEM. Except with probability at most ϵ , decapsulation of an honestly generated ciphertext either returns the encapsulated key or returns $\perp$. Since a completed initiator session cannot have output $\perp$, it follows that

$$K_d = K_e$$

outside the rigidity-bad event.

It remains to show that the accepted authentication messages correspond to the matching peer and to the same key-exchange transcript. In the actual protocol, the data-to-be-signed strings are the role-separated strings

$$\text{DTBS}_I = \text{'0'} \parallel \text{sid} \parallel ek \parallel ct_{\text{kem}},$$

and

$$\text{DTBS}_R = \text{'1'} \parallel \text{sid} \parallel ek \parallel ct_{\text{kem}}.$$

The string '0' denotes the initiator-authentication direction, and '1' denotes the responder-authentication direction. Hence the signatures bind the role of the signer, the session identifier, the ephemeral KEM public key, and the KEM ciphertext.

The peer identities are bound separately by the MAC layer under session-specific directional MAC keys:

$$\tau_I = \text{MAC}(mk_I, \text{'0'} \parallel \text{sid} \parallel ID_I), \quad \tau_R = \text{MAC}(mk_R, \text{'1'} \parallel \text{sid} \parallel ID_R).$$

Thus, except with probability bounded by the EUF-CMA advantages of **SIG** and **MAC**, an honest completed session accepts only an authentication message generated for the same role-separated signed string and for the intended post-specified identity.

Since both parties then evaluate the deterministic chained key schedule on the same KEM shared secret and the same nonce-derived salt,

$$(SS, mk_I, mk_R) \leftarrow \text{KDF}^+(K, N_I \parallel N_R),$$

they output the same session key SS . This proves P1, except with probability bounded by the rigidity error, authentication-forgery probabilities, and negligible collision events for honestly generated nonces and session identifiers.

P2: session-key secrecy. We now prove session-key secrecy. We use the standard CK guessing technique: the simulator guesses the tested session and its post-specified peer, embeds the IND-CPAF challenge into the guessed KEM exchange, and then compares the real and random Test branches by a sequence of hybrid games.

The proof proceeds in four steps. First, we define a standard CK simulator and a family of guessed simulators. Second, we show that, conditioned on a correct guess, the guessed tested execution is well formed except for the authentication-forgery and rigidity-bad events. Third, we prove four adjacent hybrid transitions:

$$\hat{\mathcal{S}}\text{-REAL} \approx \hat{\mathcal{S}}\text{-RPRF} \approx \hat{\mathcal{S}}\text{-ALLR} \approx \hat{\mathcal{S}}\text{-HYBR} \approx \hat{\mathcal{S}}\text{-RAND}.$$

Finally, we combine these transitions with the CK guessing loss to obtain the claimed P2 bound.

We first define the simulators used in the secrecy proof.

- (1) **The standard CK simulator.** Let $\mathcal{S}$ be the standard CK simulator for an execution of LOOM-AKE with adversary $\mathcal{A}$. The simulator maintains the full protocol state for all honest parties and answers $\mathcal{A}$'s oracle queries as follows.
 - (a) Upon a **Send** query, $\mathcal{S}$ honestly executes the protocol on behalf of the addressed party, provided that this party is not corrupted.
 - (b) Upon a **Corrupt** query to a party, $\mathcal{S}$ gives $\mathcal{A}$ the full internal information of that party, including its long-term signing key, the states of all incomplete sessions, and the session keys of all completed unexpired sessions. From that point on, $\mathcal{S}$ stops executing protocol actions on behalf of this corrupted party.
 - (c) Upon a **Reveal-state** query to an incomplete session, $\mathcal{S}$ returns the complete internal state of that session, including any unerased ephemeral KEM state.
 - (d) Upon a valid **Reveal-session-key** query to a completed unexpired session, $\mathcal{S}$ returns the session key output by that session.
 - (e) Upon a valid **Test** query, $\mathcal{S}$ answers according to the hidden test bit of the CK experiment.
 - (f) When $\mathcal{A}$ terminates, $\mathcal{S}$ terminates with the same output bit as $\mathcal{A}$.
- (2) **The guessed simulators.** We next define a family of guessed simulators, denoted generically by $\hat{\mathcal{S}}$. Such a simulator is identical to $\mathcal{S}$ on all sessions except for one guessed session and its matching peer session, if such a matching session is activated.
- (3) **Guessing the tested session and the post-specified peer.** Let N_s be an a-priori upper bound on the number of sessions activated by $\mathcal{A}$, and let N_p be the number of parties. At the beginning of the experiment, $\hat{\mathcal{S}}$ chooses uniformly at random a session index

$$t \leftarrow_{\$} \{1, \dots, N_s\}$$

and a peer identity among the N_p parties.

Let GUESS be the event that the t -th activated session is the session on which $\mathcal{A}$ eventually makes its valid **Test** query and that the guessed peer identity is the post-specified peer output by the tested session. Hence,

$$\Pr[\text{GUESS}] \geq \frac{1}{N_s N_p}.$$

If GUESS does not occur, or if $\mathcal{A}$ makes no valid **Test** query, then $\hat{\mathcal{S}}$ aborts.

- (4) **Embedding the IND-CPAF challenge.** Conditioned on GUESS, the simulator embeds the IND-CPAF challenge into the guessed KEM exchange. The challenge public key is used as the ephemeral KEM public key in the relevant initiator start message, and the challenge ciphertext is used as the KEM ciphertext in the corresponding responder flight. More concretely, if the guessed tested session is an initiator-side session, then the challenge public key is placed in the start message of that session, and the challenge ciphertext is placed in the matching responder-side response. If the guessed tested session is a responder-side session, then the challenge public key is placed in the matching initiator-side start message, and the challenge ciphertext is placed in the responder flight of the tested session.
- (5) **Simulating decapsulation failures.** The simulator uses the IND-CPAF failure oracle to reproduce the single public accept/reject bit generated by decapsulation under the

challenge ephemeral decapsulation key. If the failure oracle indicates failure, the simulated initiator-side session aborts and the ephemeral decapsulation state is erased. If the failure oracle indicates success, the simulator continues the protocol execution using the key material specified by the current hybrid game.

This simulation is valid because LOOM-AKE enforces single-use ephemeral decapsulation: each ephemeral decapsulation key is used to process at most one KEM ciphertext and is erased immediately after that attempt.

- (6) **Simulation outside the guessed embedding.** All sessions outside the guessed embedding are simulated honestly. All signatures are generated using the corresponding honest parties' signing keys, unless the party has been corrupted, in which case the adversary controls that party. The directional MAC tags in the guessed session are generated and verified using the MAC keys specified by the current hybrid game.

Having fixed the generic guessed simulator $\hat{\mathcal{S}}$ and the embedding of the IND-CPAF challenge, it remains to specify how $\hat{\mathcal{S}}$ derives or samples the key material in the guessed test session. This key material consists of the internal session key SS_{int}^* , the directional MAC keys mk_I^*, mk_R^* , and the value SS_{test}^* returned by the Test oracle. The following five games instantiate $\hat{\mathcal{S}}$ with different choices of these values, while keeping the rest of the simulation unchanged. Let

$$\text{salt} := N_I \parallel N_R,$$

For a KDF key x , write

$$\begin{aligned} SS_x &= \text{KDF}(x, \text{salt} \parallel 0\text{x}01), \\ mk_{I,x} &= \text{KDF}(x, SS_x \parallel \text{salt} \parallel 0\text{x}02), \\ mk_{R,x} &= \text{KDF}(x, mk_{I,x} \parallel \text{salt} \parallel 0\text{x}03). \end{aligned}$$

Equivalently,

$$(SS_x, mk_{I,x}, mk_{R,x}) \leftarrow \text{KDF}^+(x, \text{salt}).$$

Because the first KDF output SS is also used as an input to the derivation of mk_I , we distinguish the internal KDF output SS_{int}^* from the value SS_{test}^* returned by the Test oracle. In the real branch these two values are identical, whereas in the random branch only SS_{test}^* is replaced by an independent random string. The internal protocol computation, including the computation and verification of MAC tags, continues to use the internally derived keys.

Conditioned on GUESS, the proof proceeds through five games:

$$\begin{aligned} \hat{\mathcal{S}}\text{-REAL} : & (SS_{\text{int}}^*, mk_I^*, mk_R^*) \leftarrow \text{KDF}^+(K', \text{salt}), & SS_{\text{test}}^* &:= SS_{\text{int}}^*, \\ \hat{\mathcal{S}}\text{-RPRF} : & k \leftarrow \$ \mathcal{K}_{\text{kdf}}, \\ & (SS_{\text{int}}^*, mk_I^*, mk_R^*) \leftarrow \text{KDF}^+(k, \text{salt}), & SS_{\text{test}}^* &:= SS_{\text{int}}^*, \\ \hat{\mathcal{S}}\text{-ALLR} : & SS_{\text{int}}^* \leftarrow \$ \mathcal{K}_{\text{sess}}, & mk_I^* &\leftarrow \$ \mathcal{K}_{\text{mac}}, & mk_R^* &\leftarrow \$ \mathcal{K}_{\text{mac}}, & SS_{\text{test}}^* &:= SS_{\text{int}}^*, \\ \hat{\mathcal{S}}\text{-HYBR} : & k \leftarrow \$ \mathcal{K}_{\text{kdf}}, \\ & (SS_{\text{int}}^*, mk_I^*, mk_R^*) \leftarrow \text{KDF}^+(k, \text{salt}), & SS_{\text{test}}^* &\leftarrow \$ \mathcal{K}_{\text{sess}}, \\ \hat{\mathcal{S}}\text{-RAND} : & (SS_{\text{int}}^*, mk_I^*, mk_R^*) \leftarrow \text{KDF}^+(K', \text{salt}), & SS_{\text{test}}^* &\leftarrow \$ \mathcal{K}_{\text{sess}}. \end{aligned}$$

Here K' denotes the KEM shared secret associated with the guessed test session: on the responder side it is the encapsulated key K_e , and on the initiator side it is the decapsulated key K_d when decapsulation succeeds. By rigidity, any accepting guessed test session satisfies $K_d = K_e$ except with probability at most ϵ .

The first hop, $\hat{\mathcal{S}}\text{-REAL} \approx \hat{\mathcal{S}}\text{-RPRF}$, replaces the real KEM shared secret K' by an independently sampled KDF key $k \leftarrow \$ \mathcal{K}_{\text{kdf}}$ before applying the same chained KDF^+ schedule. This hop

is justified by IND-CPAF security of the ephemeral KEM, because the simulator can use the IND-CPAF failure oracle to reproduce the protocol's accept/reject behavior for the challenge ciphertext.

The second hop, $\hat{\mathcal{S}}\text{-RPRF} \approx \hat{\mathcal{S}}\text{-ALLR}$, replaces the outputs of the chained KDF schedule under a random key k by independent uniform values. This hop is justified by the PRF security of KDF and by the canonical, domain-separated encodings of the inputs $\text{salt} \parallel 0\text{x}01$, $SS_{\text{int}}^* \parallel \text{salt} \parallel 0\text{x}02$, and $mk_I^* \parallel \text{salt} \parallel 0\text{x}03$.

The third hop, $\hat{\mathcal{S}}\text{-ALLR} \approx \hat{\mathcal{S}}\text{-HYBR}$, restores the internal protocol keys to the output of KDF^+ under an independent random KDF key k , while keeping the value returned by the Test oracle uniformly random. This hop is again justified by PRF security. This step is necessary because the CK Test query replaces only the tested session key, not the authentication keys used to simulate the protocol execution.

The fourth hop, $\hat{\mathcal{S}}\text{-HYBR} \approx \hat{\mathcal{S}}\text{-RAND}$, replaces the independent KDF key k used in the internal protocol computation by the real KEM secret K' . This hop is justified by IND-CPAF security. The final game $\hat{\mathcal{S}}\text{-RAND}$ is exactly the random branch of the CK Test experiment: the protocol execution, including MAC computation and verification, uses the real KEM-derived internal keys, whereas the Test oracle returns an independent random session key SS_{test}^* .

Therefore, in $\hat{\mathcal{S}}\text{-RAND}$ the value returned by the Test query is independent of the adversary's view. Any non-negligible distinction between the real and random Test branches yields either an IND-CPAF distinguisher, a PRF distinguisher, or an authentication forgery. Accounting for the CK guessing probability gives the claimed P2 bound.

Before proving the hybrid transitions, we record the structural properties of a fresh tested execution under the guessed embedding. This lemma is not a hybrid step; rather, it ensures that, conditioned on GUESS, the simulators above do not abort except with probability accounted for by authentication-forgery events, the rigidity error, and negligible collision events for honestly generated nonces and session identifiers.

Lemma 4.14 (Good test session under the guessed embedding). *For all LOOM-AKE adversaries $\mathcal{A}$, conditioned on the event GUESS that the simulator correctly guesses the tested session and its post-specified peer, the following properties hold except with probability bounded by the authentication-forgery probabilities and negligible terms.*

A. *Consider a regular execution in which $\mathcal{A}$ chooses a tested session with output (P, s, Q) , where P is the initiator. Then:*

- (1) *P and Q are not corrupted before expiration of the tested session.*
- (2) *The tested session and its matching session, if any, are not revealed by $\mathcal{A}$.*
- (3) *The responder-side session at Q is activated using the start message generated by the initiator-side session (P, s) .*
- (4) *The initiator-side session (P, s) receives a response message containing the same KEM ciphertext ct_{kem} as the one output by the responder-side session at Q .*
- (5) *The tested session accepts and does not abort.*
- (6) *The KEM exchange in the tested session is good: except with probability at most ϵ , if the tested session accepts then $K_d = K_e$, and the KEM shared secret associated with the guessed session is indistinguishable from a uniformly random KEM key even conditioned on the allowed IND-CPAF failure leakage.*

B. *Consider a regular execution in which $\mathcal{A}$ chooses a tested session with output (Q, s, P) , where Q is the responder. Then:*

- (1) P and Q are not corrupted before expiration of the tested session.
- (2) The tested session and its matching session, if any, are not revealed by $\mathcal{A}$.
- (3) The initiator-side session at P has sent the start message used by the responder-side tested session at Q .
- (4) The responder-side tested session receives a start message carrying the same ephemeral KEM public key ek as the one generated by the initiator-side session at P .
- (5) The tested session accepts and does not abort.
- (6) The KEM exchange in the tested session is good in the same sense as in Part A.

Proof. Items (1) and (2) follow directly from the freshness condition for the Test query in the CK experiment. Item (5) follows from the fact that the Test query is valid only for an accepted session.

It remains to justify the matching properties in Items (3) and (4), and the KEM-goodness property in Item (6). Consider first the initiator-tested case in Part A. If an uncorrupted initiator accepts a responder authentication message that was not generated by the matching responder session for the same key-exchange transcript, then the adversary must have caused acceptance of a valid authentication message outside the honest matching execution. In the actual protocol, the responder's data-to-be-signed string is

$$\text{DTBS}_R = \text{'1'} \parallel \text{sid} \parallel ek \parallel ct_{\text{kem}}.$$

Thus, producing a valid responder authentication message for a new role-separated signed string yields an EUF-CMA forgery against SIG. If the signed string is not new, but the accepted post-specified identity is not the intended one, then the adversary must have produced a valid MAC tag of the form

$$\tau_R = \text{MAC}(mk_R, \text{'1'} \parallel \text{sid} \parallel ID_R)$$

under the session-specific responder-direction MAC key mk_R . This yields a MAC forgery. Therefore, except with probability bounded by the signature and MAC forgery advantages, the accepted responder authentication message corresponds to the matching responder-side execution and to the same ek, ct_{kem} , and the intended responder identity.

The responder-tested case in Part B is analogous. If an uncorrupted responder accepts an initiator authentication message not generated by the matching initiator session, then the adversary must either forge a signature on

$$\text{DTBS}_I = \text{'0'} \parallel \text{sid} \parallel ek \parallel ct_{\text{kem}},$$

or forge a MAC tag

$$\tau_I = \text{MAC}(mk_I, \text{'0'} \parallel \text{sid} \parallel ID_I)$$

under the corresponding initiator-direction MAC key mk_I . Hence the matching properties hold except with probability bounded by the corresponding authentication-forgery advantages.

Finally, Item (6) follows from ϵ -rigidity and IND-CPAF security of the KEM. By ϵ -rigidity, except with probability at most ϵ , decapsulation of an honestly generated ciphertext either returns the encapsulated key or returns $\perp$. Since an accepting tested session cannot have returned $\perp$, it follows that $K_d = K_e$ outside the rigidity-bad event. Moreover, the single-use decapsulation rule guarantees that the adversary sees at most one public accept/reject bit for the challenge ephemeral decapsulation key. This is exactly the failure leakage allowed in the IND-CPAF experiment. Therefore the KEM shared secret associated with the guessed session remains indistinguishable from a uniformly random KEM key under the IND-CPAF assumption. This proves the lemma. $\square$

We now prove the four adjacent hybrid transitions. Since Lemma 4.14 has isolated the bad events, each transition changes only the key material used in the guessed session. The first and fourth transitions are reduced to IND-CPAF security of the KEM, while the second and third transitions are reduced to PRF security of the KDF.

Lemma 4.15. *For all LOOM-AKE adversaries $\mathcal{A}$, conditioned on GUESS,*

$$\hat{\mathcal{S}}\text{-REAL}(\mathcal{A}) \approx \hat{\mathcal{S}}\text{-RPRF}(\mathcal{A}),$$

assuming IND-CPAF security of the underlying KEM.

Proof. The two games differ only in the key used as input to the chained KDF^+ computation for the guessed test session. In $\hat{\mathcal{S}}\text{-REAL}$, the simulator uses the real KEM shared secret K' associated with the guessed session. In $\hat{\mathcal{S}}\text{-RPRF}$, it uses an independently sampled KDF key $k \leftarrow \$ \mathcal{K}_{\text{kdf}}$.

We construct an IND-CPAF distinguisher $\mathcal{B}_1$. The distinguisher receives an IND-CPAF challenge consisting of an ephemeral KEM public key, a challenge ciphertext, and a challenge key, denoted

$$(ek^*, ct^*, K_b).$$

It embeds ek^* as the ephemeral KEM public key in the guessed initiator-side session and embeds ct^* as the KEM ciphertext in the corresponding responder flight. The nonces N_I, N_R are generated as in the real protocol, and the salt is set to

$$salt := N_I \parallel N_R.$$

When the initiator side processes the challenge ciphertext, the simulator uses the IND-CPAF failure oracle to reproduce the public accept/reject behavior of decapsulation. If the oracle indicates failure, the simulated session aborts. If it indicates success, the simulator continues the protocol execution.

For the guessed test session, the simulator computes

$$(SS_{\text{int}}^*, mk_I^*, mk_R^*) \leftarrow \text{KDF}^+(K_b, salt), \quad SS_{\text{test}}^* := SS_{\text{int}}^*.$$

The internal keys mk_I^*, mk_R^* are used to generate and verify the MAC tags in the guessed session. All signatures and all other protocol messages are simulated honestly. All sessions outside the guessed embedding are also simulated honestly.

If K_b is the real KEM shared secret, then the simulation is identical to $\hat{\mathcal{S}}\text{-REAL}$, except for the rigidity-bad event already accounted for in Lemma 4.14. If K_b is an independently random KEM key, then the simulation is identical to $\hat{\mathcal{S}}\text{-RPRF}$, because the chained KDF^+ computation is applied to an independent key. Therefore, any non-negligible distinguisher between $\hat{\mathcal{S}}\text{-REAL}$ and $\hat{\mathcal{S}}\text{-RPRF}$ yields a non-negligible IND-CPAF distinguisher against the KEM. $\square$

Lemma 4.16. *For all LOOM-AKE adversaries $\mathcal{A}$, conditioned on GUESS,*

$$\hat{\mathcal{S}}\text{-RPRF}(\mathcal{A}) \approx \hat{\mathcal{S}}\text{-ALLR}(\mathcal{A}),$$

assuming PRF security of the keyed function family induced by KDF.

Proof. In $\hat{\mathcal{S}}\text{-RPRF}$, the simulator samples $k \leftarrow \$ \mathcal{K}_{\text{kdf}}$ and computes

$$\begin{aligned} SS_{\text{int}}^* &= \text{KDF}(k, salt \parallel 0x01), \\ mk_I^* &= \text{KDF}(k, SS_{\text{int}}^* \parallel salt \parallel 0x02), \\ mk_R^* &= \text{KDF}(k, mk_I^* \parallel salt \parallel 0x03), \end{aligned}$$

and sets

$$SS_{\text{test}}^* := SS_{\text{int}}^*.$$

In $\hat{\mathcal{S}}\text{-ALLR}$, the simulator instead samples

$$SS_{\text{int}}^* \leftarrow \$ \mathcal{K}_{\text{sess}}, \quad mk_I^* \leftarrow \$ \mathcal{K}_{\text{mac}}, \quad mk_R^* \leftarrow \$ \mathcal{K}_{\text{mac}},$$

independently, and again sets

$$SS_{\text{test}}^* := SS_{\text{int}}^*.$$

We build a PRF distinguisher $\mathcal{B}_2$ that simulates the guessed test session and answers the above KDF evaluations using its oracle $F(\cdot)$. The three queries made by the simulator are

$$\text{salt} \parallel 0\mathbf{x}01, \quad SS_{\text{int}}^* \parallel \text{salt} \parallel 0\mathbf{x}02, \quad mk_I^* \parallel \text{salt} \parallel 0\mathbf{x}03.$$

These inputs are canonically encoded and domain separated. In particular, the suffixes $0\mathbf{x}01, 0\mathbf{x}02, 0\mathbf{x}03$ and the injective encoding of the variable-length fields ensure that the three oracle inputs are distinct with probability one.

If F is a real KDF instance under a random key k , the simulation is exactly $\hat{\mathcal{S}}\text{-RPRF}$. If F is a uniformly random function, then its outputs on the three distinct inputs are independent uniform values in the corresponding output spaces, giving exactly $\hat{\mathcal{S}}\text{-ALLR}$. Hence any non-negligible distinction between $\hat{\mathcal{S}}\text{-RPRF}$ and $\hat{\mathcal{S}}\text{-ALLR}$ yields a non-negligible PRF advantage against KDF. $\square$

Lemma 4.17. *For all LOOM-AKE adversaries $\mathcal{A}$, conditioned on GUESS,*

$$\hat{\mathcal{S}}\text{-ALLR}(\mathcal{A}) \approx \hat{\mathcal{S}}\text{-HYBR}(\mathcal{A}),$$

assuming PRF security of the keyed function family induced by KDF.

Proof. The purpose of this hop is to restore the internal protocol keys to the output of the chained key schedule under an independent random KDF key, while keeping the value returned by the Test oracle uniformly random.

In $\hat{\mathcal{S}}\text{-ALLR}$, the simulator samples

$$SS_{\text{int}}^* \leftarrow \$ \mathcal{K}_{\text{sess}}, \quad mk_I^* \leftarrow \$ \mathcal{K}_{\text{mac}}, \quad mk_R^* \leftarrow \$ \mathcal{K}_{\text{mac}},$$

and sets

$$SS_{\text{test}}^* := SS_{\text{int}}^*.$$

Since SS_{int}^* is uniform, this is distributionally equivalent, with respect to the value returned by the Test oracle, to first sampling an independent

$$SS_{\text{test}}^* \leftarrow \$ \mathcal{K}_{\text{sess}}$$

and then sampling independent uniform internal authentication keys.

In $\hat{\mathcal{S}}\text{-HYBR}$, the simulator samples $k \leftarrow \$ \mathcal{K}_{\text{kdf}}$ and computes the internal protocol keys by

$$(SS_{\text{int}}^*, mk_I^*, mk_R^*) \leftarrow \text{KDF}^+(k, \text{salt}),$$

while sampling

$$SS_{\text{test}}^* \leftarrow \$ \mathcal{K}_{\text{sess}}$$

independently for the Test response.

We reduce the distinction between these two games to PRF security. A PRF distinguisher $\mathcal{B}_3$ simulates the guessed test session and uses its oracle $F(\cdot)$ to answer the chained KDF evaluations

$$\text{salt} \parallel 0\mathbf{x}01, \quad SS_{\text{int}}^* \parallel \text{salt} \parallel 0\mathbf{x}02, \quad mk_I^* \parallel \text{salt} \parallel 0\mathbf{x}03.$$

The Test response is always an independently sampled $SS_{\text{test}}^* \leftarrow \$ \mathcal{K}_{\text{sess}}$.

If F is a real KDF under an independent random key k , the internal protocol keys are distributed as in $\hat{\mathcal{S}}\text{-HYBR}$. If F is a uniformly random function, then the internal outputs are independent uniform values, and the simulation is distributed as $\hat{\mathcal{S}}\text{-ALLR}$, up to the purely notational separation between SS_{int}^* and SS_{test}^* . The oracle inputs are distinct by the same domain-separation and injective-encoding argument as in Lemma 4.16. Therefore, any non-negligible distinguisher between $\hat{\mathcal{S}}\text{-ALLR}$ and $\hat{\mathcal{S}}\text{-HYBR}$ yields a non-negligible PRF distinguisher. $\square$

Lemma 4.18. *For all LOOM-AKE adversaries $\mathcal{A}$, conditioned on GUESS,*

$$\hat{\mathcal{S}}\text{-HYBR}(\mathcal{A}) \approx \hat{\mathcal{S}}\text{-RAND}(\mathcal{A}),$$

assuming IND-CPAF security of the underlying KEM.

Proof. The two games differ only in the key used for the internal chained KDF^+ computation in the guessed test session. In $\hat{\mathcal{S}}\text{-HYBR}$, the internal protocol keys are derived from an independent random KDF key $k \leftarrow \$ \mathcal{K}_{\text{kdf}}$. In $\hat{\mathcal{S}}\text{-RAND}$, they are derived from the real KEM shared secret K' of the guessed session. In both games, the Test oracle returns an independent random session key $SS_{\text{test}}^* \leftarrow \$ \mathcal{K}_{\text{sess}}$.

We construct an IND-CPAF distinguisher $\mathcal{B}_4$ exactly as in Lemma 4.15. The distinguisher embeds the IND-CPAF challenge public key and ciphertext as the ephemeral KEM public key ek^* and KEM ciphertext ct^* in the guessed session. It uses the IND-CPAF failure oracle to simulate the public accept/reject behavior of the initiator-side decapsulation. If decapsulation succeeds, it computes the internal protocol keys as

$$(SS_{\text{int}}^*, mk_I^*, mk_R^*) \leftarrow \text{KDF}^+(K_b, \text{salt}),$$

where K_b is the IND-CPAF challenge key, and it always answers the Test query with an independent random value

$$SS_{\text{test}}^* \leftarrow \$ \mathcal{K}_{\text{sess}}.$$

If K_b is the real KEM shared secret, the simulation is exactly $\hat{\mathcal{S}}\text{-RAND}$, except for the rigidity-bad event. If K_b is uniformly random, the simulation is exactly $\hat{\mathcal{S}}\text{-HYBR}$. Therefore any non-negligible distinction between $\hat{\mathcal{S}}\text{-HYBR}$ and $\hat{\mathcal{S}}\text{-RAND}$ yields a non-negligible IND-CPAF distinguisher against the KEM. $\square$

Having established the four adjacent transitions, we combine them to prove session-key secrecy. The following lemma applies the triangle inequality to the hybrid chain, removes the conditioning on GUESS using the CK guessing loss, and adds the bad-event probabilities from Lemma 4.14.

Lemma 4.19. *Protocol LOOM-AKE satisfies condition P2 of SK-security.*

Proof. We use the guessed simulators and the five games defined above. Let GUESS be the event that the simulator correctly guesses the tested session and its post-specified peer. Since the adversary activates at most N_s sessions among N_p parties, we have

$$\Pr[\text{GUESS}] \geq \frac{1}{N_s N_p}.$$

Conditioned on GUESS, Lemma 4.14 shows that the guessed tested execution is well formed, fresh, and KEM-good, except with probability bounded by the authentication-forgery probabilities, the rigidity error ϵ , and negligible collision events for honestly generated nonces and session identifiers. Outside these bad events, the simulator can embed the IND-CPAF challenge consistently into the guessed KEM exchange and does not abort.

The four hybrid-transition lemmas give the following indistinguishability chain:

$$\hat{\mathcal{S}}\text{-REAL} \approx \hat{\mathcal{S}}\text{-RPRF} \approx \hat{\mathcal{S}}\text{-ALLR} \approx \hat{\mathcal{S}}\text{-HYBR} \approx \hat{\mathcal{S}}\text{-RAND}.$$

More precisely, Lemma 4.15 bounds the first transition by an IND-CPAF advantage of the KEM, Lemma 4.16 bounds the second transition by a PRF advantage of KDF, Lemma 4.17 bounds the third transition by another PRF advantage of KDF, and Lemma 4.18 bounds the fourth transition by another IND-CPAF advantage of the KEM.

The game $\hat{\mathcal{S}}\text{-REAL}$ is the real branch of the CK Test experiment for the guessed session, because the Test oracle returns

$$SS_{\text{test}}^* = SS_{\text{int}}^*.$$

The game $\hat{\mathcal{S}}\text{-RAND}$ is the random branch of the CK Test experiment, because the internal protocol execution still uses the real KEM-derived keys

$$(SS_{\text{int}}^*, mk_I^*, mk_R^*) \leftarrow \text{KDF}^+(K', \text{salt}),$$

whereas the Test oracle returns an independent random session key

$$SS_{\text{test}}^* \leftarrow \$ \mathcal{K}_{\text{sess}}.$$

Thus, in $\hat{\mathcal{S}}\text{-RAND}$ the Test response is independent of the adversary's view.

By the triangle inequality over the four hybrid transitions and by the CK guessing argument, any non-negligible advantage of $\mathcal{A}$ in distinguishing the real Test branch from the random Test branch gives rise to either an IND-CPAF distinguisher, a PRF distinguisher, or an authentication forgery, except for the rigidity-bad event and negligible collision events. Therefore P2 holds. $\square$

Proof of Theorem 4.13. P1 was established in the preceding key-consistency argument. P2 follows from Lemma 4.19. The two IND-CPAF hybrid transitions give adversaries $\mathcal{B}_1$ and $\mathcal{B}_4$ against the KEM, the two PRF hybrid transitions give adversaries $\mathcal{B}_2$ and $\mathcal{B}_3$ against KDF, and the authentication-bad events give adversaries $\mathcal{B}_5$ and $\mathcal{B}_6$ against SIG and MAC, respectively.

Accounting for the CK guessing loss, for suitable efficient adversaries $\mathcal{B}_1, \dots, \mathcal{B}_6$ we obtain

$$\begin{aligned} \text{Adv}_{\text{LOOM-AKE}}^{\text{SK}}(\mathcal{A}) &\leq N_s N_p \cdot \left(\text{Adv}_{\text{KEM}}^{\text{IND-CPAF}}(\mathcal{B}_1) + \text{Adv}_{\text{KDF}}^{\text{PRF}}(\mathcal{B}_2) \right. \\ &\quad \left. + \text{Adv}_{\text{KDF}}^{\text{PRF}}(\mathcal{B}_3) + \text{Adv}_{\text{KEM}}^{\text{IND-CPAF}}(\mathcal{B}_4) \right) \\ &\quad + \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\mathcal{B}_5) + \text{Adv}_{\text{MAC}}^{\text{MAC}}(\mathcal{B}_6) + \epsilon + \text{negl}(\lambda). \end{aligned}$$

All terms are negligible under the assumptions of the theorem. Hence LOOM-AKE is SK-secure in the post-specified-peer CK model. $\square$

4.2 Concrete Security Estimates

Our concrete security estimates are based on the Core-SVP model [5], a conservative cost model in which the time complexity of the BKZ algorithm with block size β is approximated by a single call to an SVP oracle in dimension β . For large β , sieving is the most efficient method for solving SVP in dimension β : the best known classical and quantum sieving complexities are $2^{0.292\beta}$ [8] and $2^{0.257\beta}$ [15], respectively.

4.2.1 Security Analysis of the KEM Component

The KEM component of LOOM-AKE is built from a module-lattice-based PKE. Its security relies on the hardness of the Module Learning With Rounding (MLWR) problem, which is converted to an equivalent MLWE instance by modeling the rounding noise as additive uniform noise. When estimating the concrete cost of key recovery in LOOM (i.e., solving the underlying Search-MLWR instances), the adversary can choose to attack those defined by the public key—i.e., recovering the long-term secret $\mathbf{s}$ —or those defined by the ciphertext—i.e., recovering the ephemeral secret $\mathbf{r}$ from the ciphertext c_1 and c_2 (see Lines 20 and 21 in Algorithm 11). The second ciphertext component c_2 is compressed at a coarser precision $d_v < d_u$ and is perturbed by a message-dependent shift before rounding, so it does not constitute a strict MLWR equation for $\mathbf{r}$. Nevertheless, to be conservative, we assume c_2 has the same noise level as the c_1 -based samples and treat it as an additional MLWR sample. Consequently, the ciphertext provides approximately $k + 1$ MLWR samples about $\mathbf{r}$ according to our assumption, compared to k samples about $\mathbf{s}$ from the public key [17]. Since $d_t \neq d_u$ for all parameter sets of LOOM-KEM (see Table 5), the rounding errors introduced by compression in the public key and the ciphertext differ, and the CBD parameters η_1 and η_2 also differ. Both instances are evaluated as equivalent MLWE instances by modeling the rounding error as additive uniform noise, and we take the minimum across both as the final security level.

We evaluate the cost of the main known attack families against the MLWE instance: primal attack [5, 3], BDD (hybrid) attacks [32, 2, 30], and dual attacks [14, 41]. Other approaches such as BKW attack [1], algebraic attack [6] and combinatorial attacks [42, 38] are excluded as they are far less competitive in the parameter regime of LOOM. To evaluate the hardness of MLWE, one usually first transforms the MLWE instance into a standard LWE instance and then assesses the concrete computational complexity. The study in [30] exploits the ring structure of $\mathbb{Z}_q[X]/(X^n + 1)$ to speed up BDD hybrid attacks [32, 2] and the dual attack in [14].

We note that the MATZOV dual attack [41] relies on several controversial assumptions [22]. The work of Carrier et al. [14] provides a refined analysis and experimental results that support cost estimates close to those claimed in [41], without relying on the contested assumptions. Following the approach of [24], we also consider it prudent to include the dual attacks from both [14, 41] in our cost estimates, notwithstanding the controversial assumptions underlying [41].

For the dual attack in [14], we adapted the evaluation code of [14] to accommodate our algorithm’s parameters—in particular, to handle the near-uniform distribution of error distribution in LOOM-KEM. We further observed that the ring structure can be even more fully exploited within the dual attack framework of [14] compared to the analysis in [30], and that the asymmetry between the standard deviations of the secret distribution and error distribution provides additional advantage for accelerating the dual attack. We incorporated these observations into our evaluation code.

Table 6 lists the individual MLWE attack costs for public-key-based recovery (recover $\mathbf{s}$) under the Core-SVP model, obtained using the lattice estimator [4]¹ together with our adapted evaluation code from [14]. Table 7 lists the corresponding costs for ciphertext-based recovery (recover $\mathbf{r}$). For all parameter sets, the attack costs in Table 7 are higher than those in Table 6, indicating that public-key-based recovery is the more efficient attack path.

4.2.2 Security Analysis of the SIG Component

The SIG component of LOOM-AKE is instantiated with the Shuttle signature scheme [16]. Its security evaluation accounts for both the MLWE hardness of recovering the signing key from the public key and the MSIS hardness of the signature scheme.

¹<https://github.com/malb/lattice-estimator>.

Table 6: Attack costs for public-key-based recovery (recover $\mathbf{s}$) against the KEM component’s underlying MLWR instance in the Core-SVP model.

Cost model	Primal	BDD	Dual [41]	Dual [14]
Loom-KEM-128				
Classical	165.6	166.6	159.4	159.3
Quantum	145.7	146.7	141.6	150.8
Loom-KEM-256				
Classical	278.6	279.4	281.9	267.6
Quantum	245.2	246.1	250.9	239.7
Loom-KEM-512				
Classical	568.5	569.3	535.5	530.7
Quantum	500.4	501.1	476.5	488.2

Table 7: Attack costs for ciphertext-based recovery (recover $\mathbf{r}$) against the KEM component’s underlying MLWR instance in the Core-SVP model.

Cost model	Primal	BDD	Dual [41]	Dual [14]
Loom-KEM-128				
Classical	177.5	178.4	168.5	167.3
Quantum	156.3	157.1	150.9	158.2
Loom-KEM-256				
Classical	290.8	291.8	276.5	274.4
Quantum	256.0	257.0	246.2	248.5
Loom-KEM-512				
Classical	608.8	609.8	611.9	564.6
Quantum	535.8	536.9	545.1	530.1

MLWE Costs of the SIG Component. The MLWE hardness of the SIG component is evaluated following the same methodology as the KEM component, including the same attack families and dual attack considerations [14, 41]. The parameter sets of the SIG component use Gaussian-distributed secrets and errors with standard deviations σ_1 and σ_2 , rather than the centered binomial distributions used by the KEM component. The ring structure of $\mathbb{Z}_q[X]/(X^n+1)$ can be exploited similarly, and the asymmetry between σ_1 and σ_2 in the SIG component’s parameter sets provides additional advantage for the dual attack [14].

Table 8 lists the individual MLWE attack costs for the SIG component under the Core-SVP model.

Table 8: Attack costs against the SIG component’s underlying MLWE instance in the Core-SVP model.

Cost model	Primal	BDD	Dual [41]	Dual [14]
Shuttle-128				
Classical	144.2	145.2	138.5	138.0
Quantum	127.0	127.9	123.4	133.1
Shuttle-256				
Classical	300.8	301.8	333.7	279.7
Quantum	264.7	265.7	297.9	252.0
Shuttle-512				
Classical	661.7	662.5	∞	625.0
Quantum	582.4	583.2	∞	553.4

MSIS Costs of the SIG Component. For the MSIS problem, one often transforms the MSIS instance into SIS instances to solve [?]. Solving the SIS problem under the Euclidean norm is reduced to finding a short vector in a q -ary lattice. The lattice estimator carries out this evaluation automatically: it determines the minimum BKZ block size required to find a vector of the specified length, and then estimates the attack cost in the Core-SVP model.

Strong unforgeability (sUF) prevents the adversary from producing a valid signature for an unqueried message, or a different valid signature for a previously queried message. The latter case translates to finding an MSIS solution with norm bound twice the verification bound (i.e., $2 \cdot B_v$) [18]. Since $2B_v < q$ for all parameter sets, the specialized attack of [21] for finding short integer solutions when the bound is large does not apply to our parameters. The MSIS-sUF security estimates for the SIG component in Table 9 are evaluated using the lattice estimator [4] with this relaxed bound.

Table 9: MSIS-sUF attack costs for the SIG component in the Core-SVP model.

Scheme	Classical	Quantum
Shuttle-128	165.3	145.5
Shuttle-256	266.3	234.4
Shuttle-512	522.4	459.8

4.2.3 Summary

The overall concrete security of LOOM-AKE is determined by the minimum security level across all components. For the KEM component, the security is taken as the minimum between the public-key-based recovery costs (Table 6) and the ciphertext-based recovery costs (Table 7). Table 10 summarizes the security estimates.

Table 10: Summary of concrete security estimates for LOOM-AKE (values in classical / quantum bits).

Scheme	KEM (MLWE)	SIG (MLWE)	SIG (MSIS-sUF)
LOOM-AKE-128	159.3 / 141.6	138.0 / 123.4	165.3 / 145.5
LOOM-AKE-256	267.6 / 239.7	279.7 / 252.0	266.3 / 234.4
LOOM-AKE-512	530.7 / 476.5	625.0 / 553.4	522.4 / 459.8

4.3 Side-Channel Resistance and Implementation Security

Timing attacks The majority of LOOM can follow the constant-time implementation guidance established for SHUTTLE and WEAVER [16, 17]. In particular, since LOOM no longer relies on error-correction codes, there is no need to implement BCH decoding in constant time. Consequently, a constant-time implementation can be obtained by following the implementation techniques developed for SHUTTLE and WEAVER. Decapsulation failures and explicit rejection have already been accounted for in the security analysis of LOOM, and therefore do not introduce additional timing side channels. Developers should nevertheless ensure that the algorithms described in Section 2 are implemented without introducing any secret-dependent branches. As in WEAVER [17], this discussion excludes timing leakage caused by variable-latency CPU instructions (e.g., KyberSlash [9]); implementers should independently verify that the target instruction set does not introduce such micro-architectural timing hazards.

Data-dependent Power/EM leakage Like most post-quantum cryptographic schemes, implementations of LOOM without dedicated countermeasures are expected to be vulnerable to power and electromagnetic (EM) side-channel attacks. Besides adopting the countermeasures discussed for SHUTTLE and WEAVER [16, 17], implementers should carefully ensure that the data flow described in Section 2 does not introduce unintended data-dependent leakage from intermediate computations or memory accesses.

4.4 Other Security Properties

In this section, we discuss two critical security properties of the LOOM-AKE primitive: Perfect Forward Secrecy (PFS) and its security behavior under ephemeral key reuse.

Perfect Forward Secrecy (PFS). LOOM-AKE achieves Perfect Forward Secrecy for fresh expired sessions in the post-specified-peer CK model [13], following the same proof strategy as the signature-based IKE/SIGMA analysis. The long-term signing keys sk_I and sk_R are used only to authenticate the handshake transcript via the signature scheme SIG; no value derived from sk_I or sk_R is used as an input to the KEM computation or to the key-derivation function.

The secrecy of the established session key is based on the one-time KEM shared secret K generated in the ephemeral KEM exchange. The initiator generates a fresh KEM key pair (ek, dk) in `AKE.Init`, and the responder encapsulates a fresh secret to ek in `AKE.Respond`. After decapsulation and key derivation, the initiator erases dk and all intermediate KEM values, while the responder erases its encapsulation randomness and the raw KEM secret K after deriving the session keys. Thus, after the session has expired, the public transcript alone does not allow an adversary to reconstruct K .

Consider an adversary that corrupts a party only after the target session has expired and obtains the party’s long-term signing key. Since the corresponding ephemeral KEM state of that session has already been erased, the corruption does not reveal dk , the encapsulation randomness, or the raw shared secret K . Moreover, the long-term signing key only enables the adversary to produce future signatures; it does not help to recover the past KEM secret

from the recorded ciphertext ct_{kem} . By the IND-CPAF security of the ephemeral KEM and the pseudorandomness of the key-derivation function, the historical session key SS remains computationally indistinguishable from a random string.

This PFS claim is subject to the standard freshness conditions of the CK model: the adversary must not have issued a reveal-state query for the target session or its matching session before completion, and must not have obtained the session key through a reveal-session-key query. Under these conditions, post-expiration compromise of long-term signing keys does not compromise past session keys.

Behavior Under Key Reuse (Strict Ephemeral Requirement): A defining feature of LOOM-KEM is that it omits the computationally expensive ciphertext re-encryption step of the Fujisaki–Okamoto (FO) transform, relying instead on IND-CPAF security and rigidity. While this optimization yields a $4\times$ speedup in decapsulation, it introduces a strict architectural constraint: **the ephemeral KEM keys must never be reused across multiple sessions.**

If the KEM key pair (ek, dk) is reused (i.e., if ek is treated as a static or semi-static key):

1. **Failure Oracle Attacks:** An active adversary can initiate multiple handshakes using the same public key ek and inject manipulated ciphertexts ct'_i . By observing whether the connection aborts or retries (which acts as an explicit Failure Oracle due to our rigidity check), the adversary can collect binary feedback on whether decryption succeeded.
2. **Key Recovery:** Because LOOM-KEM has a non-negligible decryption failure rate (DFR) and lacks FO re-encryption, an adversary can exploit these failure boundaries. Using established techniques such as those by D’Anvers et al. [19] or belief propagation [28], the adversary can reconstruct the secret key dk after observing only a small number of decryption failures.

To mitigate this risk, the implementation of LOOM-AKE must strictly enforce the following operational safeguards:

- **Strict Single-Use:** Every session initiator must sample a fresh ephemeral key pair $(ek, dk) \leftarrow \text{KEM.KGen}(1^\lambda)$ for each handshake. Reusing ek for subsequent connection attempts is strictly prohibited.
- **Immediate Erasure:** The decapsulation key dk must be zeroized in memory immediately after the decapsulation function $\text{Decap}(dk, ct_{\text{kem}})$ is executed, regardless of whether the decapsulation succeeds or outputs $\perp$.
- **State Machine Enforcement:** The protocol state machine must ensure that any incoming ciphertext is processed at most once per ephemeral state.

Under these strict ephemeral guarantees, the adversary can query the Failure Oracle at most once per key pair, which bounds the geometric leakage to exactly 1 bit of information. As proven in Theorem 4.13, this 1-bit leakage is fully neutralized by the IND-CPAF security reduction and the KDF extraction, ensuring robust session-key security.

5 Performance Evaluation and Implementation Analysis

This section presents a comprehensive evaluation of *Loom-AKE* performance on x86_64. We report median CPU cycles for the composed LOOM-KEM KEM and *Shuttle* signature modules, for each of the six normative AKE algorithms (§2.2), and for end-to-end session establishment. All numbers in §5.2 were collected in one benchmark session on Intel Core i7-14700 (see §5.1).

5.1 Evaluation Methodology

To ensure reproducibility, we define the x86_64 benchmarking environment as follows:

- **Platform (x86_64):** Intel Core i7-14700 (20 cores, 28 threads), 16 GB RAM; benchmarks run on Ubuntu 24.04 under WSL2.
 - *Compiler:* `gcc 13.3.0` (Ubuntu 13.3.0-6ubuntu2 24.04.1)
 - *Reference implementation:* portable ISO C99 (`-O3 -march=native`), SHAKE-256 symmetric branch (`-DLOOM_USE_SHAKE=ON`)
 - *Optimized implementation:* separate AVX2 tree (`avx2/`) compiled with `-O3 -march=native -mavx2 -mbmi2 -mpopcnt -mfma`
- **Metric definition.** Performance is reported as median CPU cycles measured via the processor time-stamp counter (RDTSCP). Before each timed block we execute $N_{\text{warmup}}=100$ untimed iterations; we then record $N_{\text{test}}=10,000$ timed samples, subtract a per-call timing overhead estimated from empty RDTSCP pairs, sort the samples, and take the median. Each table entry therefore comes from a *single* run of 10,000 measurements per algorithm—we do *not* repeat independent benchmark sessions and average medians.

5.2 Software Performance on x86 Processors

We provide two x86_64 implementations: a portable ISO C99 reference build and an optimized build that vectorizes the LOOM-KEM/Shuttle polynomial layer with AVX2. Section 2.2 defines six normative AKE algorithms; four emit on-wire messages (`msgI,1`–`msgR,2`), while **KeyGen** and **Confirm** perform credential setup and final verification without producing a handshake message. The subsections below report primitive KEM and signature cost first, then composed AKE steps, and finally a timed full-handshake total.

5.2.1 Loom-KEM: Reference vs. AVX2

Table 11 compares isolated LOOM-KEM primitives on Intel Core i7-14700. AVX2 yields consistent speedups at all three parameter sets: **1.6**×–**2.6**× on encapsulation and decapsulation, with the largest gains at Loom-AKE-256 where $q=7681$ NTT and parallel matrix expansion dominate key generation and encapsulation. Decapsulation remains the cheapest primitive in every row because the rigid decaps path avoids full re-encryption; even so, the optimized build reduces decaps to under 14,000 cycles at the 256-bit level.

Table 11: LOOM-KEM primitive performance (CPU cycles, Intel Core i7-14700; median of $N_{\text{test}}=10,000$ samples)

Parameter set	Implementation	KeyGen	Encaps	Decaps
Loom-AKE-128	Reference (C99)	63,262	72,589	13,692
	Optimized (AVX2)	34,524	46,379	7,725
Loom-AKE-256	Reference (C99)	100,073	137,371	36,350
	Optimized (AVX2)	42,859	54,010	13,889
Loom-AKE-512	Reference (C99)	234,354	324,065	67,466
	Optimized (AVX2)	128,897	200,252	40,852

Across parameter sets, **Init** and **Respond** in Table 13 align closely with these entries because passes 1 and 2 are dominated by LOOM-KEM key generation, encapsulation, and decapsulation;

the absolute cost remains small relative to Shuttle work in Table 12 and the authentication columns of Table 13.

5.2.2 Shuttle signature: Reference vs. AVX2

Table 12 reports isolated Shuttle costs. Signature operations are substantially more expensive than KEM at every level, and achieve the largest gains under AVX2: speedups range from **2.5× to 3.3×**, with key generation and verification improving by roughly 3× at the 128- and 256-bit levels. Signing remains the most costly single primitive (up to 2.26 M cycles reference at Loom-AKE-512), reflecting NTT-dominated rounding, parallel XOF sampling, and rejection logic in the Shuttle profile.

Table 12: Shuttle signature primitive performance (CPU cycles, Intel Core i7-14700; median of $N_{\text{test}}=10,000$ samples)

Parameter set	Implementation	KeyGen	Sign	Verify
Loom-AKE-128	Reference (C99)	597,668	577,089	130,862
	Optimized (AVX2)	196,633	220,163	48,838
Loom-AKE-256	Reference (C99)	923,248	1,039,513	213,430
	Optimized (AVX2)	277,035	347,835	72,120
Loom-AKE-512	Reference (C99)	1,739,517	2,255,134	433,552
	Optimized (AVX2)	599,319	891,715	133,551

Each session performs two long-term Shuttle key generations (one per party) and two signed authentication passes. Comparing Tables 12 and 13, **AuthInit** and **AuthResp** each cost nearly an order of magnitude more than **Init** and **Respond**, respectively, and **KeyGen** exceeds the LOOM-KEM passes by several-fold; optimizing Shuttle therefore has the largest impact on practical *Loom-AKE* performance.

5.2.3 Loom-AKE per-algorithm cost

Table 13 maps §2.2 Algorithms 1–6 to isolated median cycles. The same timing harness as Tables 11–12 is used, but each column times *one* normative step only (prior passes run as untimed setup). **KeyGen**, **AuthInit**, and **AuthResp** align with the signature-intensive rows in Table 12; **Init** and **Respond** align with LOOM-KEM encaps/decaps workloads; **Confirm** covers initiator-side signature verification and MAC checking and excludes session finalization.

We observe two trends. First, authentication passes constitute the largest share of per-step cost: at Loom-AKE-128 (reference), **AuthInit+AuthResp** exceed **Init+Respond** by nearly one order of magnitude ($\approx 9\times$). Second, AVX2 preserves the relative ranking—**KeyGen** and the two signed passes remain the dominant components—while delivering roughly 2.5–3.3× speedups on Shuttle-dominated columns and about 1.7–1.8× on **Init** and **Respond**. These columns must *not* be summed to approximate session cost: **KeyGen** counts one party only, later columns include untimed setup through preceding passes, and **Confirm** omits **finalize**. Use Table 14 for end-to-end totals.

5.2.4 End-to-end session cost

Table 14 reports the only additive end-to-end figure in this section: median CPU cycles for a complete timed handshake (both parties’ long-term **KeyGen**, all four messages, shared-secret derivation, and session teardown). Primitive and per-algorithm tables above use isolated timing

Table 13: Loom-AKE per-algorithm cost (CPU cycles, Intel Core i7-14700; median of $N_{\text{test}}=10,000$ samples; isolated timing, not additive). Second header row: on-wire message (“—” if none).

Parameter set	Impl.	AKE algorithm (wire message)					
		KeyGen —	Init $\text{msg}_{I,1}$	Respond $\text{msg}_{R,1}$	AuthInit $\text{msg}_{I,2}$	AuthResp $\text{msg}_{R,2}$	Confirm —
Loom-AKE-128	Ref.	626,614	73,072	87,859	641,525	781,698	149,420
	AVX2	225,129	40,646	51,526	261,130	316,725	59,516
Loom-AKE-256	Ref.	929,680	105,917	138,854	1,131,969	1,342,873	229,245
	AVX2	302,898	50,018	59,669	427,782	499,570	88,642
Loom-AKE-512	Ref.	1,817,397	236,791	331,042	2,391,147	2,692,192	447,903
	AVX2	589,650	136,458	197,034	1,002,737	1,162,061	168,650

and must not be summed to approximate this total. AVX2 delivers **2.58**×–**2.64**× speedups across the three parameter sets (3.54 M→ 1.34 M cycles at Loom-AKE-128; 4.68 M→ 1.81 M at Loom-AKE-256; 10.74 M→ 4.14 M at Loom-AKE-512).

Table 14: Loom-AKE end-to-end session cost (CPU cycles, Intel Core i7-14700; median of $N_{\text{test}}=10,000$ full-handshake samples; same session as Tables 11–13)

Parameter set	Reference total	AVX2 total	Speedup
Loom-AKE-128	3,538,956	1,339,335	2.64 ×
Loom-AKE-256	4,682,385	1,812,218	2.58 ×
Loom-AKE-512	10,741,650	4,136,594	2.60 ×

Taken together, Tables 11–14 show that Shuttle signature work accounts for the largest share of *Loom-AKE* session time at all security levels, while AVX2 accelerates the full handshake by 2.58×–2.64× without materially shifting this cost composition.

5.3 Optimization Techniques

To achieve the performance in §5.2, we apply AVX2-specific optimizations on top of the portable ISO C99 reference baseline.

5.3.1 Vectorization with AVX2 (x86_64)

The AVX2 tree keeps the Keccak/SHAKE symmetric layer aligned with ML-KEM (SHAKE128 matrix XOF, SHAKE256 PRFs) and vectorizes the polynomial layer.

Loom-KEM (KEM).

- **Modulus-specific NTT kernels.** At $q=3329$ we reuse a Kyber-style AVX2 NTT/basemul stack (`ntt3329_avx128`, $n=128$ for LOOM-KEM-128). At $q=7681$ we provide dedicated AVX2 forward/inverse NTT and basemul kernels for LOOM-KEM-256 ($n=256$) and LOOM-KEM-512 ($n=512$), operating in standard coefficient order (no `nttunpack` on the hot path). Butterflies use `vpmullw/vpmulhw` Montgomery multiplication with staged (lazy) reduction between layers.
- **Parallel matrix expansion.** `gen_matrix/gen_matrixT` are implemented with $\text{SHAKE128} \times 4$ (`keccak4x/`): four XOF streams are absorbed and squeezed in parallel via the Keccak-

$\text{f}[1600] \times 4$ SIMD256 sponge, and rejection sampling fills matrix entries concurrently. This is the largest single kernel after NTT on $q=7681$ modes.

- **Vectorized rejection sampling.** For $q=3329$, `rej_uniform_avx` processes 32 candidates per iteration with `vpshufb` unpacking and `movemask` compaction; tails fall back to a scalar path. For $q=7681$, Lemire-style rejection uses vectorized 8-lane stores inside the $\times 4$ matrix generator (`rej_uniform7681`).
- **Fused module arithmetic.** `polyvec_basemul_acc_montgomery` accumulates the rank- k inner product in NTT domain; each coefficient block calls the AVX2 `basemul*` kernel, reducing $\text{NTT} \rightarrow \text{basemul} \rightarrow \text{accumulate}$ traffic for key generation and encapsulation.
- **AVX2 compression and (de)serialization.** Public-key and ciphertext packing use width-specific AVX2 helpers: 9-bit compression for LOOM-KEM-128 ($d_t=9$), 10-bit for LOOM-KEM-256 ($d_t=10$), and 11-bit for LOOM-KEM-512 ($d_t=11$), plus scheme-specific (d_u, d_v) ciphertext bit widths from Table 2. `msgenc.c` shares the same compress/decompress AVX2 paths for the message polynomial.
- **AVX2 centered binomial sampling (CBD).** For $\eta \in \{2, 3, 7\}$ we vectorize CBD (Kyber-style $\eta=2/3$; SWAR 32-bit popcount for $\eta=7$). For $\eta=9$ (LOOM-KEM-512) the reference bit-extraction path is retained: end-to-end CBD cost is still dominated by SHAKE256 PRF expansion, so the remaining gap vs. ML-KEM is primarily parameterization ($\eta_1=9$ vs. 2), not missing SIMD in the sampler core.

Shuttle (signature). Shuttle constitutes a large share of end-to-end session cost—long-term key generation plus the two signed authentication exchanges. The AVX2 implementation vectorizes both the NTT-heavy rounding pipeline and the XOF-driven sampling layer:

- **Modulus-specific AVX2 NTT stack.** Each security level uses hand-tuned forward/inverse NTT, Montgomery pointwise multiply, and (for the signed $q=15361$ profile) lazy-reduction renormalization. On signing, forward transforms are cached and reused across NTT-domain matrix-vector products in the rounding path; inverse transforms feed commitment reconstruction.
- **SIMD rounding, lifting, and hint handling.** Modular centering, round-half-up quantization, signature-hint application, and freeze-and-glue loops in commitment reconstruction are vectorized branchlessly, matching the portable reference bit-for-bit.
- **Parallel SHAKE $\times 4$ XOF batching.** Matrix expansion, secret-vector expansion, and challenge-vector sampling partition work across 16 logical XOF streams; four Keccak- $\text{f}[1600]$ lanes absorb and squeeze in parallel so initial blocks refill in batched passes rather than sequential per-stream squeezes, while per-lane rejection logic stays unchanged.
- **Vectorized uniform rejection (matrix expansion).** Sixteen contiguous uniform candidates are tested per SIMD iteration; accepted lanes are compacted with BMI2 scatter/gather idioms. Matrix expansion constitutes the principal component of verification latency and remains a large fraction of signing cost.
- **AVX2 reverse-CDT base sampler.** Discrete-Gaussian magnitude draws use an 8-lane borrow-chain over grouped 96-bit reverse-CDT thresholds, processed in 16-sample tiles and verified against the scalar oracle; secret and challenge expansion inherit this path.
- **Vectorized norm checks and inverse-rejection sampling.** Squared ℓ_2 norms are accumulated eight coefficients at a time for signing and verification gates. The four full-polynomial inverse-rejection passes vectorize inner products, axpy updates, and condi-

tional negations; PRNG bytes for rejection transitions are scheduled in bulk to reduce per-transition XOF refills.

Build flags and validation. Release x86 measurements use `-O3 -march=native -mavx2 -mbmi2 -mpopcnt -mfma`; Keccak $\times 4$ and some compress/rejsample translation units are compiled with `-fno-lto` for stable linkage with hand-written assembly. Correctness is checked by byte-identical replay of reference vectors (ICCS KAT and cross-mode SHAKE KAT).

5.4 Transmission and Storage Overhead

Minimizing communication bandwidth and storage overhead is a primary design goal for practical post-quantum authenticated key exchange. *Loom-AKE* realizes ICCS-style entity authentication in four messages by composing ephemeral LOOM-KEM key establishment with mutual *Shuttle* signatures and PRF-MAC key confirmation. Table 15 summarizes the long-term credential sizes, the normative handshake message capacities, and the total serialized handshake footprint for the three parameter sets.

Table 15: Data sizes and transmission overhead of *Loom-AKE* parameter sets

Instance	$ pk $	$ sk $	$ msg_{I,1} $	$ msg_{R,1} $	$ msg_{I,2} $	$ msg_{R,2} $	Total
Loom-AKE-128	1264	2288	792	776	1235	1235	4038
Loom-AKE-256	1952	3680	1352	1384	2485	2485	7706
Loom-AKE-512	3648	7104	2952	3016	5101	5101	16170

All entries in bytes. $|pk|/|sk|$ are long-term Shuttle keys (LOOM.PKBYTES/LOOM.SKBYTES).

Total = $\sum_{k=1}^4 |msg_k|$ (handshake transmission overhead); shared secret $|ss| \in \{16, 32, 64\}$ bytes.

Message layout. Messages 1 and 2 carry the ephemeral LOOM-KEM material; messages 3 and 4 carry authenticated identities, compact Shuttle signatures, and a PRF-derived MAC. The long-term Shuttle public key is *not* embedded in the four handshake messages—it is supplied to *AuthResp/Confirm* as a peer credential (analogous to a pre-distributed certificate). Table 16 details the field decomposition; each of messages 1 and 2 begins with a fixed 8-byte SPI field ($SPI = SPI_I || SPI_R$). In $msg_{I,1}$ only SPI_I is sampled and the second half is zero (reserved for SPI_R in $msg_{R,1}$).

Table 16: Serialized layout of the four *Loom-AKE* handshake messages (bytes)

Message	Field	128	256	512
$msg_{I,1}$	SPI ($SPI_I 0^4$)	8	8	8
	Ephemeral LOOM-KEM pk_{KEM}	752	1312	2880
	Initiator nonce N_I	32	32	64
	<i>Subtotal</i>	792	1352	2952
$msg_{R,1}$	SPI ($SPI_I SPI_R$)	8	8	8
	Ephemeral LOOM-KEM ct_{KEM} (body + CPAF tag)	736	1344	2944
	Responder nonce N_R	32	32	64
	<i>Subtotal</i>	776	1384	3016
$msg_{I,2} / msg_{R,2}$	idlen (LE16) + identity (max)	34	34	34
	siglen (LE16) + Shuttle signature (max)	1185	2419	5003
	PRF-MAC tag τ	16	32	64
	<i>Subtotal</i>	1235	2485	5101

Table 15 shows that handshake cost scales with both LOOM-KEM parameter sets and Shuttle signature sizes. At Loom-AKE-128 the four messages total 4038 bytes: passes 1–2 ($\text{msg}_{I,1} + \text{msg}_{R,1}$, 1568 bytes) deliver ephemeral LOOM-KEM pk_{KEM} and ct_{KEM} together with session-binding material (SPI, N_I , N_R); passes 3–4 ($\text{msg}_{I,2} + \text{msg}_{R,2}$, 2470 bytes) carry mutual authentication (identities, two compact Shuttle signatures, and PRF-MAC tags). The authentication passes therefore account for the largest share of the serialized footprint ($\approx 61\%$ at the 128-bit level), reflecting the AKE security goal of explicit peer authentication and confirmed session keys rather than ephemeral key transport alone. The same decomposition holds at higher levels: LOOM-KEM payloads grow from 1488 to 5824 bytes across the three sets, while the per-party authentication envelope grows from 1235 to 5101 bytes.

In terms of network protocol compatibility, every Loom-AKE-128 message fits within a standard 1500-byte Ethernet MTU once IP/UDP headers are accounted for (the largest pass, $\text{msg}_{I,2}/\text{msg}_{R,2}$, is 1235 bytes). At Loom-AKE-256 the authentication messages exceed the MTU (2485 bytes) and require IP fragmentation or application-layer chunking in UDP-based deployments such as IKEv2/IPsec; at Loom-AKE-512 both the LOOM-KEM passes (2952 and 3016 bytes) and the authentication messages (5101 bytes) exceed the MTU. The LOOM-KEM passes remain fragment-friendly at the 256-bit level (1352 and 1384 bytes).

Long-term and session storage. Each party stores one long-term Shuttle key pair for all sessions; Table 17 summarizes persistent and per-session storage (values from `ref/loom/params.h` and the `LOOM_KEX_CTX` layout in `loom_loc.h`). The secret key follows the packed *Shuttle* `skEncode` format (`seedA`, embedded public polynomial body, challenge seeds, and packed secret vectors). During a handshake the implementation additionally allocates:

- an ephemeral LOOM-KEM secret share `sk_kem` (`LOOM_KEM_SKBYTES`);
- exchange buffers `self_exch` and `peer_exch`, each of `LOOM_EXCH_LENMAX = max(|pkKEM|, |ctKEM|)` bytes, together with SPI, both nonces, PRF/MAC subkeys, and derived key material (`keymat`, `ss`);
- pointers into the long-term credential (`self_privkey`, `self_pubkey`, identity) rather than second copies of $|pk|/|sk|$.

Ephemeral LOOM-KEM state is erased with `LOOM_KEX_CTX`; only the long-term Shuttle sk (and optionally pk) persists on disk. A peer Shuttle pk of the same $|pk|$ as in Table 17 is supplied at runtime and is not counted in the persistent column. Caching exchange values and nonces in the session context avoids re-parsing during later passes; stack buffers for signing/transcript hashing add $\mathcal{O}(|\text{sig}|)$ transient memory beyond Table 17.

Table 17: Local storage per party (*Loom-AKE*; bytes)

Parameter set	Persistent (disk)		Transient (active session)			$ ss $
	$ pk $	$ sk $	$ sk_{\text{KEM}} $	Session buffers [†]	Total [‡]	
Loom-AKE-128	1264	2288	960	1640	2600	16
Loom-AKE-256	1952	3680	1664	2888	4552	32
Loom-AKE-512	3648	7104	3328	6280	9608	64

[†]Session buffers = `SPI` + `NI` + `NR` + $2 \cdot \text{LOOM_EXCH_LENMAX}$ + $2 \cdot \text{LOOM_MACBYTES}$ + $2 \cdot \text{LOOM_KEM_SSBYTES}$ (embedded `LOOM_KEX_CTX` fields, excluding long-term key material).

[‡]Total transient = `LOOM_KEM_SKBYTES` + session buffers; peak RAM also includes the persistent $|pk|/|sk|$ while credentials remain loaded. $|pk|/|sk|$ match Table 15.

References

- [1] Martin R. Albrecht, Carlos Cid, Jean-Charles Faugère, Robert Fitzpatrick, and Ludovic Perret. On the complexity of the BKW algorithm on LWE. volume 74, pages 325–354, 2015.
- [2] Martin R. Albrecht, Benjamin R. Curtis, and Thomas Wunderer. Exploring trade-offs in batch bounded distance decoding. In Kenneth G. Paterson and Douglas Stebila, editors, *Selected Areas in Cryptology - SAC 2019 - 26th International Conference, Waterloo, ON, Canada, August 12-16, 2019, Revised Selected Papers*, volume 11959 of *Lecture Notes in Computer Science*, pages 467–491. Springer, 2019.
- [3] Martin R. Albrecht, Florian Göpfert, Fernando Virdia, and Thomas Wunderer. Revisiting the expected cost of solving usvp and applications to LWE. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I*, Lecture Notes in Computer Science, pages 297–322. Springer, 2017.
- [4] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *J. Math. Cryptol.*, 9(3):169–203, 2015.
- [5] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange - A new hope. In Thorsten Holz and Stefan Savage, editors, *25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016*, pages 327–343. USENIX Association, 2016.
- [6] Sanjeev Arora and Rong Ge. New algorithms for learning in presence of errors. In *International Colloquium on Automata, Languages, and Programming*, pages 403–415. Springer, 2011.
- [7] Roberto Avanzi, J Bos, L Ducas, E Kiltz, T Lepoint, V Lyubashevsky, J Schanck, P Schwabe, G Seiler, and D Stehlé. CRYSTALS-Kyber: Algorithm specifications and supporting documentation (version 3.0). Technical report, National Institute of Standards and Technology, 2020.
- [8] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. In Robert Krauthgamer, editor, *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2016, Arlington, VA, USA, January 10-12, 2016*, pages 10–24. SIAM, 2016.
- [9] Daniel J. Bernstein, Karthikeyan Bhargavan, Shivam Bhasin, Anupam Chattopadhyay, Tee Kiah Chia, Matthias J. Kannwischer, Franziskus Kiefer, Thales B. Paiva, Prasanna Ravi, and Goutam Tamvada. Kyberslash: Exploiting secret-dependent division timings in kyber implementations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(2):209–234, Mar. 2025.
- [10] Ron Bonica, Fred Baker, Geoff Huston, Robert M. Hinden, Ole Troan, and Fernando Gont. IP fragmentation considered fragile. RFC 8900, RFC Editor, September 2020.
- [11] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber: a CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367. IEEE, 2018.

- [12] Ran Canetti and Hugo Krawczyk. Analysis of key-exchange protocols and their use for building secure channels. In *International conference on the theory and applications of cryptographic techniques*, pages 453–474. Springer, 2001.
- [13] Ran Canetti and Hugo Krawczyk. Security analysis of ike’s signature-based key-exchange protocol. In *Annual international cryptology conference*, pages 143–161. Springer, 2002.
- [14] Kévin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the impact of a variant of MATZOV’s dual attack on kyber. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part I*, volume 16000 of *Lecture Notes in Computer Science*, pages 444–476. Springer, 2025.
- [15] André Chailloux and Johanna Loyer. Lattice sieving via quantum random walks. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part IV*, volume 13093 of *Lecture Notes in Computer Science*, pages 63–91. Springer, 2021.
- [16] Long Chen, Yifan Ming, Jipeng Zhang, Cong Zhang, Zihan Liu, Guofeng Tang, Si Gao, Pengfei Chen, Yutao Sun, Taiyu Wang, Minxuan Mei, and Jianhua Hou. Algorithm specification and supporting documentation: Shuttle — a lattice based digital signature scheme. NGCC Algorithm Proposal PKCSIGN-729909, Commercial Cryptography Standard Research Institute, June 2026. Submitted to the New Generation Commercial Cryptography (NGCC) Algorithm Call.
- [17] Long Chen, Zidi Zhuang, Tanping Zhou, Jianhua Hou, Si Gao, Yuyang Xiao, Yingjie Hao, Shuo Chen, and Ruiqi Mi. Algorithm specification and supporting documentation: Weaver — a lattice based key encapsulation mechanism. NGCC Algorithm Proposal PKCKEM-568232, Commercial Cryptography Standard Research Institute, June 2026. Submitted to the New Generation Commercial Cryptography (NGCC) Algorithm Call.
- [18] Jung Hee Cheon, Hyeongmin Choe, Julien Devevey, Tim Güneysu, Dongyeon Hong, Minhyeok Kang, Taekyung Kim, Jeongbeen Ko, Markus Krausz, Georg Land, Marc Möller, Junbum Shin, Damien Stehlé, and MinJune Yi. HAETAE: Shorter lattice-based Fiat–Shamir signatures. Algorithm Specification and Supporting Documentation, 2024.
- [19] Jan-Pieter D’Anvers, Qian Guo, Thomas Johansson, Alexander Nilsson, Frederik Vercauteren, and Ingrid Verbauwhede. Decryption failure attacks on IND-CCA secure lattice-based schemes. In *Public-Key Cryptography–PKC 2019: 22nd IACR International Conference on Practice and Theory of Public-Key Cryptography, Beijing, China, April 14-17, 2019, Proceedings, Part II*, pages 565–598. Springer, 2019.
- [20] Stephen E. Deering and Robert M. Hinden. Internet protocol, version 6 (IPv6) specification. RFC 8200, RFC Editor, July 2017.
- [21] Léo Ducas, Thomas Espitau, and Eamonn W Postlethwaite. Finding short integer solutions when the modulus is small. In *Annual International Cryptology Conference*, pages 150–176. Springer, 2023.
- [22] Léo Ducas and Ludo N. Pulles. Does the dual-sieve attack on learning with errors even work? In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III*, volume 14083 of *Lecture Notes in Computer Science*, pages 37–69. Springer, 2023.

- [23] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, pages 537–554, 1999.
- [24] Lewis Glabush, Patrick Longa, Michael Naehrig, Chris Peikert, Douglas Stebila, and Fernando Virdia. Frodokem: A CCA-secure learning with errors key encapsulation mechanism. *IACR Commun. Cryptol.*, 2(3):25, 2025.
- [25] Markus Grassl, Brandon Langenberg, Martin Roetteler, and Rainer Steinwandt. Applying Grover’s algorithm to AES: Quantum resource estimates. In *Post-Quantum Cryptography - 7th International Workshop, PQCrypto 2016*, pages 29–43. Springer, 2016.
- [26] Qian Guo, Thomas Johansson, and Alexander Nilsson. A generic attack on lattice-based schemes using decryption errors with application to ss-ntru-pke. *Cryptology ePrint Archive*, 2019.
- [27] Qian Guo, Thomas Johansson, and Jing Yang. A novel CCA attack using decryption errors against LAC. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 82–111. Springer, 2019.
- [28] Julius Hermelink, Erik Mårtensson, Simona Samardjiska, Peter Pessl, and Gabi Dreier. Belief propagation meets lattice reduction: Security estimates for error-tolerant key recovery from decryption errors. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(4):287–317, 2023.
- [29] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In *Theory of Cryptography - 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part I*, pages 341–371, 2017.
- [30] Jianhua Hou, Haodong Jiang, and Tabitha Ogilvie. Careful with the ring! concrete hardness gaps between LWE and MLWE. *To appear at CRYPTO 2026*, 2026.
- [31] Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Failing gracefully: decryption failures and the Fujisaki-Okamoto transform. In *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part III*, pages 414–443. Springer, 2022.
- [32] Nick Howgrave-Graham. A hybrid lattice-reduction and meet-in-the-middle attack against NTRU. In Alfred Menezes, editor, *Advances in Cryptology - CRYPTO 2007, 27th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2007, Proceedings*, volume 4622 of *Lecture Notes in Computer Science*, pages 150–169. Springer, 2007.
- [33] Nick Howgrave-Graham, Phong Q. Nguyen, David Pointcheval, John Proos, Joseph H. Silverman, Ari Singer, and William Whyte. The impact of decryption failures on the security of NTRU encryption. In *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*, pages 226–246, 2003.
- [34] Andreas Hülsing, Kai-Chun Ning, Peter Schwabe, Fiona Johanna Weber, and Philip R Zimmermann. Post-quantum wireguard. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 304–321. IEEE, 2021.

- [35] Samuel Jaques, Michael Naehrig, Martin Roetteler, and Fernando Virdia. Implementing Grover oracles for quantum key search on AES and LowMC. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 280–310. Springer, 2020.
- [36] Charlie Kaufman, Paul Hoffman, Yoav Nir, Pasi Eronen, and Tero Kivinen. Internet key exchange protocol version 2 (IKEv2). RFC 7296, RFC Editor, October 2014.
- [37] Hugo Krawczyk. Sigma: The ‘sign-and-mac’ approach to authenticated diffie-hellman and its use in the ike protocols. In *Annual international cryptology conference*, pages 400–425. Springer, 2003.
- [38] Eunmin Lee, Joohee Lee, Yongha Son, and Yuntao Wang. Improved meet-lwe attack via ternary trees. *Cryptology ePrint Archive*, 2024.
- [39] Feng-Hao Liu and Zhedong Wang. Rounding in the rings. In *Annual International Cryptology Conference*, pages 296–326. Springer, 2020.
- [40] Christian Majenz and Fabrizio Sisinni. Provable security against decryption failure attacks from LWE. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part II*, pages 456–485. Springer, 2024.
- [41] MATZOV. Report on the security of LWE: Improved dual lattice attack. 2022.
- [42] Alexander May. How to meet ternary LWE keys. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 701–731. Springer, 2021.
- [43] National Institute of Standards and Technology. Submission requirements and evaluation criteria for the post-quantum cryptography standardization process. 2016.
- [44] Valery Smyslov. Internet key exchange protocol version 2 (IKEv2) message fragmentation. RFC 7383, RFC Editor, November 2014.
- [45] Valery Smyslov. Intermediate exchange in the IKEv2 protocol. RFC 9242, RFC Editor, May 2022.